Brain Tumor Classification with Transfer Learning

This notebook studies a **multi-class MRI classification** problem for distinguishing several brain tumor categories and a normal class. The main objective is to compare transfer-learning models, justify the modeling choices, and test whether the selected models generalize to a different dataset.

Table of contents

1. Dataset selection
2. Dataset description and balancing
3. Data preprocessing
4. Model building and evaluation
5. Model comparison and model selection
6. Hyperparameter tuning
7. External testing on a second dataset
8. Final remarks and references

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

1. Dataset Selection

For this project, the selected source dataset is the **Brain Tumor Classification Dataset** hosted on Kaggle. It contains **brain MRI images** grouped into multiple diagnostic categories, which makes it suitable for a **multi-class image classification** workflow.

Classes used in the project

- Meningioma
- Astrocitoma
- NORMAL
- Schwannoma
- Neurocitoma
- Carcinoma
- Papiloma
- Oligodendroglioma
- Glioblastoma
- Ependimoma
- Tuberculoma
- Meduloblastoma
- Germinoma
- Granuloma
- Ganglioglioma

Why this dataset fits the project

This dataset is appropriate because it allows the notebook to address a realistic medical-imaging classification problem while remaining computationally manageable for transfer learning experiments. It also provides enough variation to compare different pretrained CNN backbones under the same pipeline.

Importing the required libraries

This section loads the libraries used for:

- data handling and visualization,
- image preprocessing and generators,
- model construction and training,
- evaluation, comparison, and reporting.

Keeping imports near the beginning makes the workflow easier to audit and reproduce.

```
!pip install tensorflow
import os
import re
```

```
import requests
from PIL import Image
import pandas as pd
import numpy as np

#libraries for data visualization
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
# import Deep learning Libraries
#preprocessing, modeling & Evaluation
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report,confusion_matrix
from sklearn.preprocessing import LabelEncoder
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.layers import Conv2D, BatchNormalization, MaxPool2D, Dropout, Flatten, Dense
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from tensorflow.keras.preprocessing import image
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam, Adamax
from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras import regularizers
import warnings
warnings.filterwarnings('ignore')
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorfl
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.2)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.2.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.80.0)
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensor
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.19.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tens
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tens
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorf
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow) (11.
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensor
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorf
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboa
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->t
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0-
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->kera
```

Downloading the dataset

The next code cell downloads the dataset directly from Kaggle and prepares the working path used in the rest of the notebook. This makes the notebook self-contained and easier to reproduce on a different machine or cloud environment.

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("fernando2rad/brain-tumor-mri-images-44c")

print("Path to dataset files:", path)
```

```
filepaths=[]
labels=[]


folds = os.listdir(path)
for fold in folds:
    foldpath = os.path.join(path, fold)
    filelist = os.listdir(foldpath)
    for file in filelist:
        fpath = os.path.join(foldpath, file)
        filepaths.append(fpath)
        # Select just tumor name
        fold = str(fold).replace('C+', '')
        labels.append(fold[:-3])


# Concatenate data paths with labels into one dataframe
Fseries = pd.Series(filepaths, name= 'filepaths')
Lseries = pd.Series(labels, name='labels')
df = pd.concat([Fseries, Lseries], axis= 1)
df
```

```
Using Colab cache for faster access to the 'brain-tumor-mri-images-44c' dataset.
Path to dataset files: /kaggle/input/brain-tumor-mri-images-44c
```

	filepaths	labels
0	/kaggle/input/brain-tumor-mri-images-44c/Papil...	Papiloma
1	/kaggle/input/brain-tumor-mri-images-44c/Papil...	Papiloma
2	/kaggle/input/brain-tumor-mri-images-44c/Papil...	Papiloma
3	/kaggle/input/brain-tumor-mri-images-44c/Papil...	Papiloma
4	/kaggle/input/brain-tumor-mri-images-44c/Papil...	Papiloma
...	...	...
4474	/kaggle/input/brain-tumor-mri-images-44c/Germi...	Germinoma
4475	/kaggle/input/brain-tumor-mri-images-44c/Germi...	Germinoma
4476	/kaggle/input/brain-tumor-mri-images-44c/Germi...	Germinoma
4477	/kaggle/input/brain-tumor-mri-images-44c/Germi...	Germinoma
4478	/kaggle/input/brain-tumor-mri-images-44c/Germi...	Germinoma

4479 rows × 2 columns

2. Dataset Description and Balancing

This task is a **multi-class classification** task: each MRI image is assigned one class label corresponding to a tumor type or to the normal class.

Why balancing matters

In medical image classification, an imbalanced dataset can bias the model toward the majority classes. For that reason, this notebook inspects the label distribution and applies a balancing step so that model evaluation is not dominated by only the most frequent classes.

Practical relevance

A model of this type could support:

- pre-screening workflows,
- decision support for radiology review,
- educational and research experimentation in medical AI.

Splitting the data

The code below separates the data into training, validation, and test partitions.

This decision is important because each split has a different role:

- **training set:** used to fit model parameters,
- **validation set:** used to monitor learning and compare configurations,
- **test set:** used only for final evaluation.

Using separate splits reduces the risk of reporting an overly optimistic result.

```
# valid and test dataframe# Stratified split to preserve class proportions
train_df, temp_df = train_test_split(
    df,
    train_size=0.80,
    shuffle=True,
    random_state=123,
    stratify=df["labels"]
)

valid_df, test_df = train_test_split(
    temp_df,
    train_size=0.60,   # 60% of 20% = 12%
    shuffle=True,
    random_state=123,
    stratify=temp_df["labels"]
)

print("Train shape:", train_df.shape)
print("Valid shape:", valid_df.shape)
print("Test shape :", test_df.shape)

print("\nClass distribution:")
print("Train:\n", train_df["labels"].value_counts())
print("\nValid:\n", valid_df["labels"].value_counts())
print("\nTest:\n", test_df["labels"].value_counts())
```

```
Train shape: (3583, 2)
Valid shape: (537, 2)
Test shape : (359, 2)

Class distribution:
Train:
 labels
Meningioma           699
Astrocitoma          464
_NORMAL              417
Schwannoma           372
Neurocitoma          366
Carcinoma            201
Papiloma             190
Oligodendroglioma    179
Glioblastoma         163
Ependimoma           120
Tuberculoma          116
Meduloblastoma       105
Germinoma             80
Granuloma             62
Ganglioglioma         49
Name: count, dtype: int64

Valid:
 labels
Meningioma           105
Astrocitoma           69
_NORMAL               63
Schwannoma            56
Neurocitoma           54
Carcinoma             30
Papiloma              28
Oligodendroglioma     27
Glioblastoma          25
Ependimoma            18
Tuberculoma           17
Meduloblastoma        16
Germinoma             12
Granuloma             10
Ganglioglioma          7
Name: count, dtype: int64

Test:
 labels
Meningioma           70
Astrocitoma          47
_NORMAL              42
Neurocitoma          37
Schwannoma           37
Carcinoma            20
Papiloma             19
Oligodendroglioma    18
Glioblastoma         16
Ependimoma           12
Tuberculoma          12
Meduloblastoma       10
Germinoma             8
```

```
import os
import tempfile
import pandas as pd
from tensorflow.keras.preprocessing.image import ImageDataGenerator

batch_size = 64
img_size = (224, 224)
channels = 3
img_shape = (img_size[0], img_size[1], channels)

# Generator used only to create synthetic images for minority classes
aug_gen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Generators for model input
train_datagen = ImageDataGenerator()
test_datagen = ImageDataGenerator()

temp_dir = tempfile.mkdtemp()
augmented_dfs = []

# Count classes ONLY in the training set
class_counts_before = train_df['labels'].value_counts()
target_count = 250

# Augment only underrepresented classes in train_df
classes_to_augment = [cls for cls, count in class_counts_before.items() if count < target_count]

print("Classes to augment:", classes_to_augment)

for class_name in classes_to_augment:
    current_class_df = train_df[train_df['labels'] == class_name].copy()
    num_existing = len(current_class_df)
    num_needed = target_count - num_existing

    if num_needed <= 0:
        continue

    class_temp_dir = os.path.join(temp_dir, class_name.replace(" ", "_"))
    os.makedirs(class_temp_dir, exist_ok=True)

    generator = aug_gen.flow_from_dataframe(
        current_class_df,
        x_col='filepaths',
        y_col='labels',
        target_size=img_size,
        class_mode='categorical',
        batch_size=1,
        shuffle=True,
        save_to_dir=class_temp_dir,
        save_prefix='aug',
        save_format='jpg'
    )

    saved_files = set()

    while len(saved_files) < num_needed:
        next(generator)
        current_files = [
            os.path.join(class_temp_dir, f)
            for f in os.listdir(class_temp_dir)
            if f.lower().endswith(".jpg")
        ]
        saved_files.update(current_files)

    saved_files = list(saved_files)[:num_needed]

    augmented_df = pd.DataFrame({
        'filepaths': saved_files,
        'labels': [class_name] * len(saved_files)
    })

    augmented_dfs.append(augmented_df)
```

```
# Final training dataframe = original training split + synthetic train-only samples
if augmented_dfs:
    final_train_df = pd.concat([train_df] + augmented_dfs, ignore_index=True)
else:
    final_train_df = train_df.copy()

class_counts_after = final_train_df['labels'].value_counts()

if len(class_counts_after) > 1:
    top_class = class_counts_after.idxmax()
    top_count = class_counts_after.max()
    second_top_count = class_counts_after.sort_values(ascending=False).iloc[1]

    if top_count > second_top_count:
        print(f"Reducing class '{top_class}' from {top_count} to {second_top_count}")
        keep_df = final_train_df[final_train_df['labels'] != top_class]
        top_class_df = final_train_df[final_train_df['labels'] == top_class].sample(
            n=second_top_count,
            random_state=42
        )
        final_train_df = pd.concat([keep_df, top_class_df], ignore_index=True)

print("\nFinal training class counts:")
print(final_train_df['labels'].value_counts())

# Build generators
train_gen = train_datagen.flow_from_dataframe(
    final_train_df,
    x_col='filepaths',
    y_col='labels',
    target_size=img_size,
    class_mode='categorical',
    batch_size=batch_size,
    shuffle=True
)

valid_gen = test_datagen.flow_from_dataframe(
    valid_df,
    x_col='filepaths',
    y_col='labels',
    target_size=img_size,
    class_mode='categorical',
    batch_size=batch_size,
    shuffle=False
)

test_gen = test_datagen.flow_from_dataframe(
    test_df,
    x_col='filepaths',
    y_col='labels',
    target_size=img_size,
    class_mode='categorical',
    batch_size=batch_size,
    shuffle=False
)
```

```
Classes to augment: ['Carcinoma', 'Papiloma', 'Oligodendroglioma', 'Glioblastoma', 'Ependimoma', 'Tuberculoma', 'Meduloblast
Found 201 validated image filenames belonging to 1 classes.
Found 190 validated image filenames belonging to 1 classes.
Found 179 validated image filenames belonging to 1 classes.
Found 163 validated image filenames belonging to 1 classes.
Found 120 validated image filenames belonging to 1 classes.
Found 116 validated image filenames belonging to 1 classes.
Found 105 validated image filenames belonging to 1 classes.
Found 80 validated image filenames belonging to 1 classes.
Found 62 validated image filenames belonging to 1 classes.
Found 49 validated image filenames belonging to 1 classes.
Reducing class 'Meningioma' from 699 to 464

Final training class counts:
labels
Astrocitoma          464
Meningioma           464
_NORMAL              417
Schwannoma           372
Neurocitoma          366
Papiloma             250
Glioblastoma         250
Carcinoma            250
Meduloblastoma       250
Germinoma            250
Tuberculoma          250
Granuloma            250
Ganglioglioma        250
Oligodendroglioma    250
```

```
Ependimoma           250
Name: count, dtype: int64
Found 4583 validated image filenames belonging to 15 classes.
Found 536 validated image filenames belonging to 15 classes.
Found 359 validated image filenames belonging to 15 classes.
```

Visualising the data

Before training a model, it is useful to inspect class examples and class counts. This helps verify that:

- labels look correct,
- classes are represented as expected,
- balancing decisions can be justified visually rather than assumed.

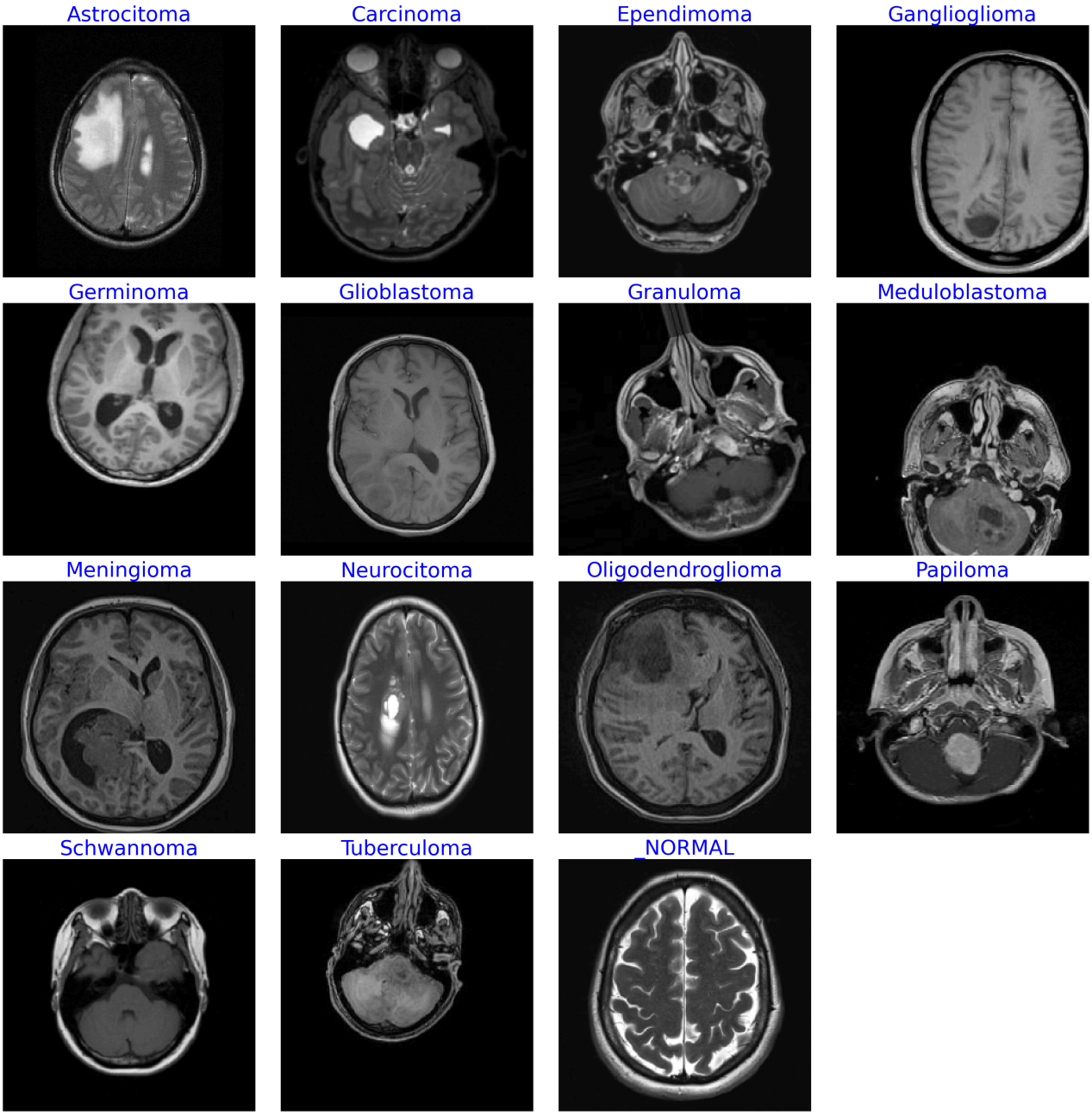
```
import matplotlib.pyplot as plt
import numpy as np

g_dict = train_gen.class_indices  # Dictionary mapping class names to indices
classes = list(g_dict.keys())     # List of class names

# Create a dictionary to store one image per class
class_images = {}

# Iterate through the generator until we have one image per class
while len(class_images) < len(classes):
    images, labels = next(train_gen)  # Get a batch of images and labels
    for i in range(images.shape[0]):
        index = np.argmax(labels[i])  # Get the class index
        class_name = classes[index]   # Get the class name
        if class_name not in class_images:
            class_images[class_name] = images[i] / 255  # Store the image

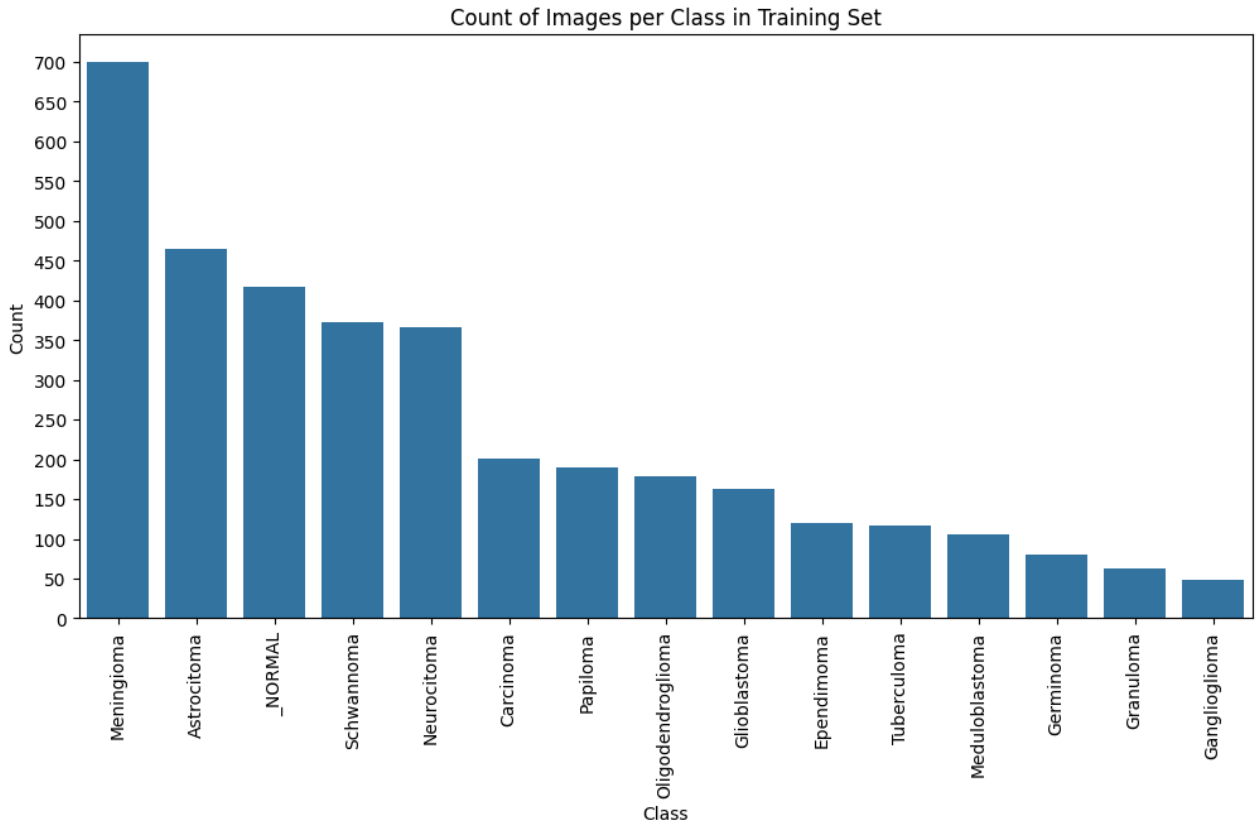
# Plot the images
plt.figure(figsize=(30, 30))
for i, class_name in enumerate(classes):
    plt.subplot(4, 4, i + 1)  # Adjust subplot layout
    plt.imshow(class_images[class_name])
    plt.title(class_name, color='blue', fontsize=40)
    plt.axis('off')
plt.tight_layout()
plt.show()
```




```
# Plot class counts using TRAINING DATA ONLY
class_counts_before = train_df['labels'].value_counts()

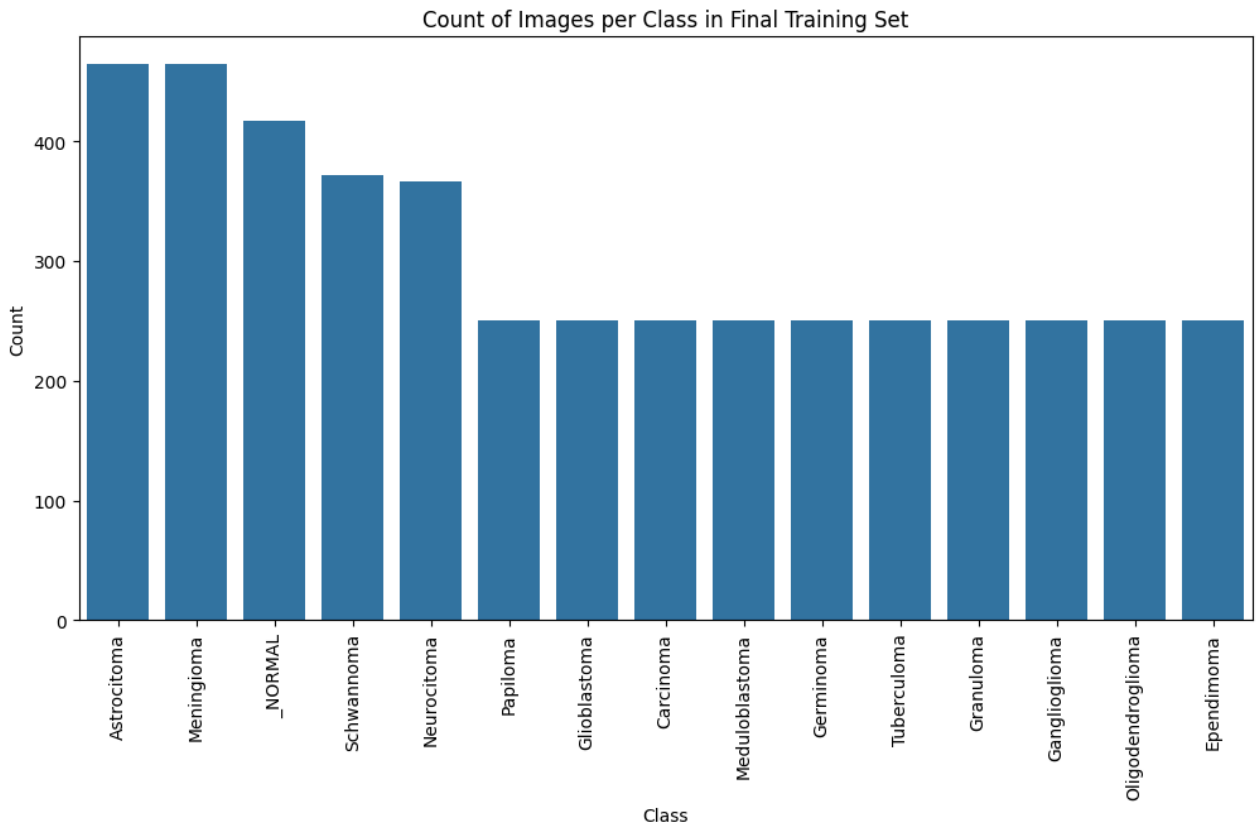
plt.figure(figsize=(12, 6))
sns.countplot(x='labels', data=train_df, order=class_counts_before.index)
plt.title('Count of Images per Class in Training Set')
```

```
plt.xlabel('Class')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.yticks(range(0, int(class_counts_before.max()) + 50, 50))
plt.show()
```




```
# Plot class counts AFTER balancing the training set
class_counts_after = final_train_df['labels'].value_counts()

plt.figure(figsize=(12, 6))
sns.countplot(x='labels', data=final_train_df, order=class_counts_after.index)
plt.title('Count of Images per Class in Final Training Set')
plt.xlabel('Class')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.show()
```



3. Data Preprocessing

Data preprocessing is the set of steps applied before training so that the input data is consistent and usable by the models.

In this notebook, preprocessing is used to:

- convert images into a common input format,
- prepare data generators for training, validation, and testing,
- keep the pipeline consistent across all compared architectures.

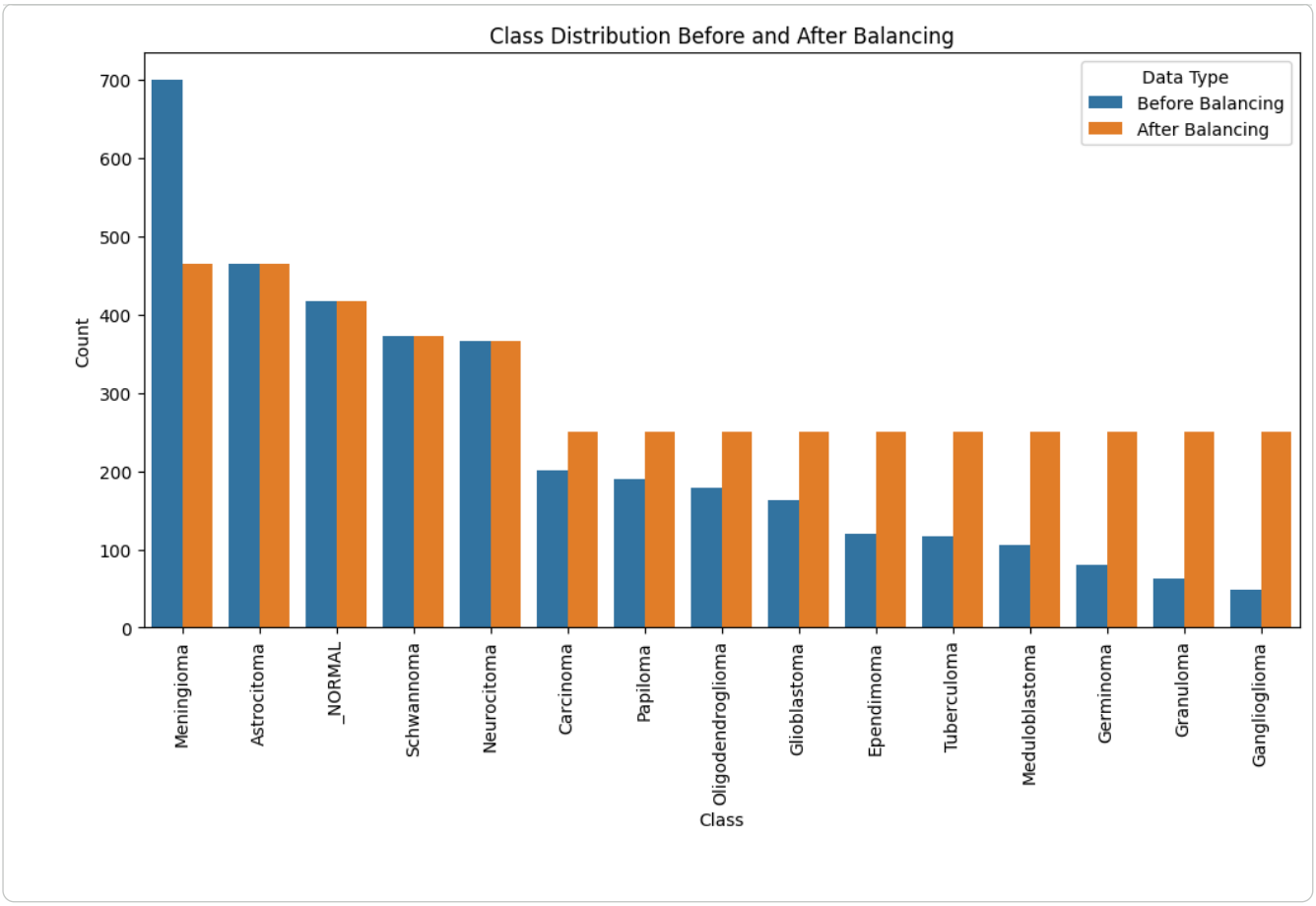
This is important because model differences should come mainly from the architectures and training decisions, not from inconsistent input handling.

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import pandas as pd



# Create a DataFrame for plotting
before_df = pd.DataFrame({'Class': class_counts_before.index, 'Count': class_counts_before.values, 'Type': 'Before Balancir
after_df = pd.DataFrame({'Class': class_counts_after.index, 'Count': class_counts_after.values, 'Type': 'After Balancing'})
plot_df = pd.concat([before_df, after_df])

# Create the grouped bar plot
plt.figure(figsize=(12, 6))
sns.barplot(x='Class', y='Count', hue='Type', data=plot_df)
plt.title('Class Distribution Before and After Balancing')
plt.xlabel('Class')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.legend(title='Data Type')
plt.show()
```



4. Model Building and Evaluation

This section implements and evaluates several transfer-learning models under the same pipeline.

Definition: Transfer learning

In deep learning, **transfer learning** means starting from a model that has already learned general visual features on a large dataset and adapting it to a new but related task.

> "Transfer learning consists of taking features learned on one problem, and leveraging them on a new, similar problem."
> — TensorFlow, *Transfer learning & fine-tuning*

Why transfer learning is used here

MRI datasets are usually much smaller than large-scale computer-vision datasets. Starting from pretrained weights is therefore a practical decision because it can improve convergence, reduce training time, and provide a stronger baseline than training from scratch.

Short research notes supporting the modeling choices

- **Transfer learning rationale:** pretrained models are especially useful when the target dataset is limited in size.
- **EfficientNet rationale:** the family is designed around compound scaling, which makes it a good candidate for comparing smaller and larger backbones.
- **DenseNet rationale:** dense connectivity promotes feature reuse and gives a structurally different baseline for comparison.

These points explain why the notebook compares EfficientNetB0, EfficientNetB5, and DenseNet121 instead of relying on only one pretrained architecture.

```
import matplotlib.pyplot as plt
import numpy as np

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.models import Model

def train_model(model, train_gen, valid_gen, epochs=10, verbose=1):
    early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
    reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2)

    history = model.fit(
```

```
            x=train_gen,
            validation_data=valid_gen,
            epochs=epochs,
            verbose=verbose,
            callbacks=[early_stop, reduce_lr],
            shuffle=True,
        )

        return history


    def plot_history(history):
        tr_acc = history.history['accuracy']
        tr_loss = history.history['loss']
        val_acc = history.history['val_accuracy']
        val_loss = history.history['val_loss']

        index_loss = np.argmin(val_loss)
        val_lowest = val_loss[index_loss]
        index_acc = np.argmax(val_acc)
        acc_highest = val_acc[index_acc]

        epochs = [i + 1 for i in range(len(tr_acc))]
        loss_label = f'best epoch= {index_loss + 1}'
        acc_label = f'best epoch= {index_acc + 1}'

        plt.figure(figsize=(20, 8))
        plt.style.use('fivethirtyeight')

        # Loss plot
        plt.subplot(1, 2, 1)
        plt.plot(epochs, tr_loss, 'r', label='Training Loss')
        plt.plot(epochs, val_loss, 'g', label='Validation Loss')
        plt.scatter(index_loss + 1, val_lowest, s=150, c='blue', label=loss_label)
        plt.title('Training and Validation Loss')
        plt.xlabel('Epochs')
        plt.ylabel('Loss')
        plt.legend()

        # Accuracy plot
        plt.subplot(1, 2, 2)
        plt.plot(epochs, tr_acc, 'r', label='Training Accuracy')
        plt.plot(epochs, val_acc, 'g', label='Validation Accuracy')
        plt.scatter(index_acc + 1, acc_highest, s=150, c='blue', label=acc_label)
        plt.title('Training and Validation Accuracy')
        plt.xlabel('Epochs')
        plt.ylabel('Accuracy')
        plt.legend()

        plt.tight_layout()
        plt.show()

    from sklearn.metrics import confusion_matrix
    import matplotlib.pyplot as plt
    import seaborn as sns
    import numpy as np

    def plot_confusion_matrix(model, generator, title="Confusion Matrix"):
        """
        Plots a confusion matrix for a given model and data generator.

        Parameters:
        - model: Trained Keras model
        - generator: Keras generator (usually test_gen)
        - title: Title for the plot
        """
        # Get true labels and predicted labels
        y_true = generator.classes
        y_pred_probs = model.predict(generator, steps=np.ceil(generator.samples / generator.batch_size).astype(int))
        y_pred = np.argmax(y_pred_probs, axis=1)

        # Compute confusion matrix
        cm = confusion_matrix(y_true, y_pred)
        class_labels = list(generator.class_indices.keys())

        # Plot
        plt.figure(figsize=(10, 8))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_labels, yticklabels=class_labels)
        plt.title(title)
        plt.xlabel('Predicted Label')
        plt.ylabel('True Label')
        plt.tight_layout()
```

```
    plt.show()

from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc, precision_score, recall_score, f1_scor
from sklearn.preprocessing import label_binarize
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import itertools

from sklearn.metrics import (
    classification_report, confusion_matrix, roc_curve, auc,
    precision_score, recall_score, f1_score, precision_recall_curve, average_precision_score
)
from sklearn.preprocessing import label_binarize
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

from sklearn.metrics import (
    classification_report, confusion_matrix, roc_curve, auc,
    precision_score, recall_score, f1_score, precision_recall_curve, average_precision_score
)
from sklearn.preprocessing import label_binarize
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

def evaluate_model_performance(model, test_gen):
    # Predict labels
    y_true = test_gen.classes
    y_pred_probs = model.predict(test_gen, steps=test_gen.samples // test_gen.batch_size + 1)
    y_pred = np.argmax(y_pred_probs, axis=1)

    # Class labels
    class_labels = list(test_gen.class_indices.keys())


    # Classification Report
    print("Classification Report:\n")
    print(classification_report(y_true, y_pred, target_names=class_labels))

    # Precision, Recall, F1 Score (macro-averaged)
    precision = precision_score(y_true, y_pred, average='macro')
    recall = recall_score(y_true, y_pred, average='macro')
    f1 = f1_score(y_true, y_pred, average='macro')

    print(f"Macro Precision: {precision:.4f}")
    print(f"Macro Recall: {recall:.4f}")
    print(f"Macro F1 Score: {f1:.4f}")

    # Binarize true labels for multi-class ROC and PR curves
    y_true_bin = label_binarize(y_true, classes=range(len(class_labels)))

    # Precision-Recall Curves per class
    precision_dict, recall_dict, pr_auc = {}, {}, {}
    for i in range(len(class_labels)):
        precision_dict[i], recall_dict[i], _ = precision_recall_curve(y_true_bin[:, i], y_pred_probs[:, i])
        pr_auc[i] = average_precision_score(y_true_bin[:, i], y_pred_probs[:, i])

    # Plot Precision-Recall curves
    plt.figure(figsize=(12, 8))
    for i in range(len(class_labels)):
        plt.plot(recall_dict[i], precision_dict[i], label=f'{class_labels[i]} (AP = {pr_auc[i]:.2f})')
    plt.title('Precision-Recall Curve per Class')
    plt.xlabel('Recall')
    plt.ylabel('Precision')
    plt.legend(loc='upper left')
    plt.grid(True)
    plt.show()
```

Model 1 — EfficientNetB0 baseline

This first model uses **EfficientNetB0** as the pretrained backbone with a custom classification head.

Research motivation

> "carefully balancing network depth, width, and resolution can lead to better performance."
> — Tan & Le, *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* (2019)

Why this model is included

EfficientNetB0 is a strong baseline because it is relatively compact while still benefiting from the EfficientNet scaling strategy. In this project, it provides a reference point for comparing a standard head, a simplified head, and a larger EfficientNet variant.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.optimizers import Adamax
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.regularizers import l2

class_count = len(train_gen.class_indices)

# Load the EfficientNetB5 base model
base_model = EfficientNetB0(
    include_top=False,
    weights='imagenet',
    input_shape=img_shape,
    pooling='max'
)

# Build the full model
efficientNet_model_b0 = Sequential([
    base_model,
    Flatten(),
     Dropout(0.5),
    Dense(256, activation='relu',kernel_regularizer=l2(0.01)),
    Dropout(0.5),
    Dense(class_count, activation='softmax')  # Final layer with softmax for multi-class
])

# Compile the model
efficientNet_model_b0.compile(
    optimizer=Adamax(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Show the model summary
efficientNet_model_b0.summary()
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16705208/16705208 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Model: "sequential"
```

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 1280)	4,049,571
flatten (Flatten)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 256)	327,936
dropout_1 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 15)	3,855

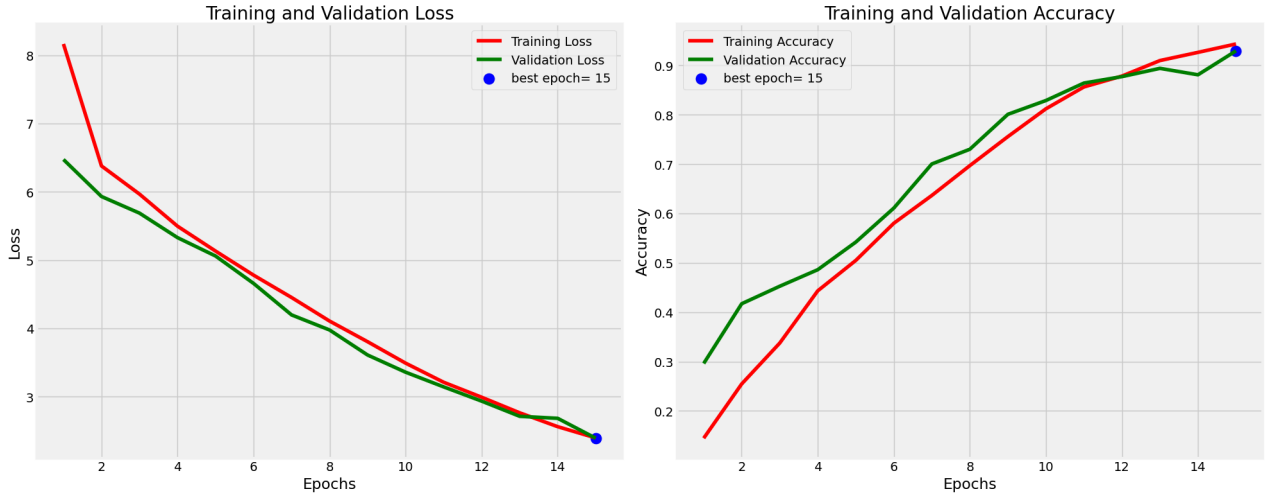
```
 Total params: 4,381,362 (16.71 MB)
 Trainable params: 4,339,339 (16.55 MB)
```

```
efficientnet_history = train_model(efficientNet_model_b0, train_gen, valid_gen, epochs=15, verbose=1)
```

```
Epoch 1/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 186s 1s/step - accuracy: 0.1447 - loss: 8.1648 - val_accuracy: 0.2961 - val_loss: 6.4734 - learni
Epoch 2/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 20s 270ms/step - accuracy: 0.2547 - loss: 6.3792 - val_accuracy: 0.4171 - val_loss: 5.9316 - lear
Epoch 3/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 21s 285ms/step - accuracy: 0.3374 - loss: 5.9666 - val_accuracy: 0.4525 - val_loss: 5.6880 - lear
Epoch 4/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 20s 272ms/step - accuracy: 0.4433 - loss: 5.4968 - val_accuracy: 0.4860 - val_loss: 5.3293 - lear
Epoch 5/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 20s 275ms/step - accuracy: 0.5048 - loss: 5.1326 - val_accuracy: 0.5419 - val_loss: 5.0602 - lear
Epoch 6/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 21s 283ms/step - accuracy: 0.5801 - loss: 4.7800 - val_accuracy: 0.6108 - val_loss: 4.6590 - lear
Epoch 7/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 20s 279ms/step - accuracy: 0.6364 - loss: 4.4527 - val_accuracy: 0.7002 - val_loss: 4.1977 - lear
Epoch 8/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 21s 296ms/step - accuracy: 0.6969 - loss: 4.1075 - val_accuracy: 0.7300 - val_loss: 3.9769 - lear
Epoch 9/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 20s 280ms/step - accuracy: 0.7556 - loss: 3.8052 - val_accuracy: 0.8007 - val_loss: 3.6104 - lear
```

```
Epoch 10/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 25s 350ms/step - accuracy: 0.8119 - loss: 3.4912 - val_accuracy: 0.8287 - val_loss: 3.3587 - lear
Epoch 11/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 23s 306ms/step - accuracy: 0.8562 - loss: 3.2118 - val_accuracy: 0.8641 - val_loss: 3.1439 - lear
Epoch 12/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 21s 296ms/step - accuracy: 0.8782 - loss: 2.9923 - val_accuracy: 0.8771 - val_loss: 2.9351 - lear
Epoch 13/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 20s 281ms/step - accuracy: 0.9096 - loss: 2.7642 - val_accuracy: 0.8939 - val_loss: 2.7130 - lear
Epoch 14/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 22s 299ms/step - accuracy: 0.9262 - loss: 2.5614 - val_accuracy: 0.8808 - val_loss: 2.6835 - lear
Epoch 15/15
72/72 ━━━━━━━━━━━━━━━━━━━━ 40s 285ms/step - accuracy: 0.9433 - loss: 2.3962 - val_accuracy: 0.9292 - val_loss: 2.3925 - lear
```

```
plot_history(efficientnet_history)
```




```
ts_length = len(test_df)
test_batch_size = max(sorted([ts_length // n for n in range(1, ts_length + 1) if ts_length%n == 0 and ts_length/n <= 80]))
#test_steps = ts_length // test_batch_size

train_score = efficientNet_model_b0.evaluate(train_gen, verbose= 1)
valid_score = efficientNet_model_b0.evaluate(valid_gen, verbose= 1)
test_score = efficientNet_model_b0.evaluate(test_gen, verbose= 1)

print("Train Loss: ", train_score[0])
print("Train Accuracy: ", train_score[1])
print('-' * 20)
print("Validation Loss: ", valid_score[0])
print("Validation Accuracy: ", valid_score[1])
print('-' * 20)
print("Test Loss: ", test_score[0])
print("Test Accuracy: ", test_score[1])
```

```
72/72 ━━━━━━━━━━━━━━━━━━━━ 19s 259ms/step - accuracy: 0.9921 - loss: 2.1610
9/9 ━━━━━━━━━━━━━━━━━━━━ 2s 195ms/step - accuracy: 0.9292 - loss: 2.3925
6/6 ━━━━━━━━━━━━━━━━━━━━ 14s 3s/step - accuracy: 0.9081 - loss: 2.4461
Train Loss:  2.1609559059143066
Train Accuracy:  0.9921431541442871
--------------------
Validation Loss:  2.3924636840820312
Validation Accuracy:  0.9292364716529846
--------------------
Test Loss:  2.4460644721984863
Test Accuracy:  0.9080780148506165
```

```
evaluate_model_performance(efficientNet_model_b0, test_gen)
```

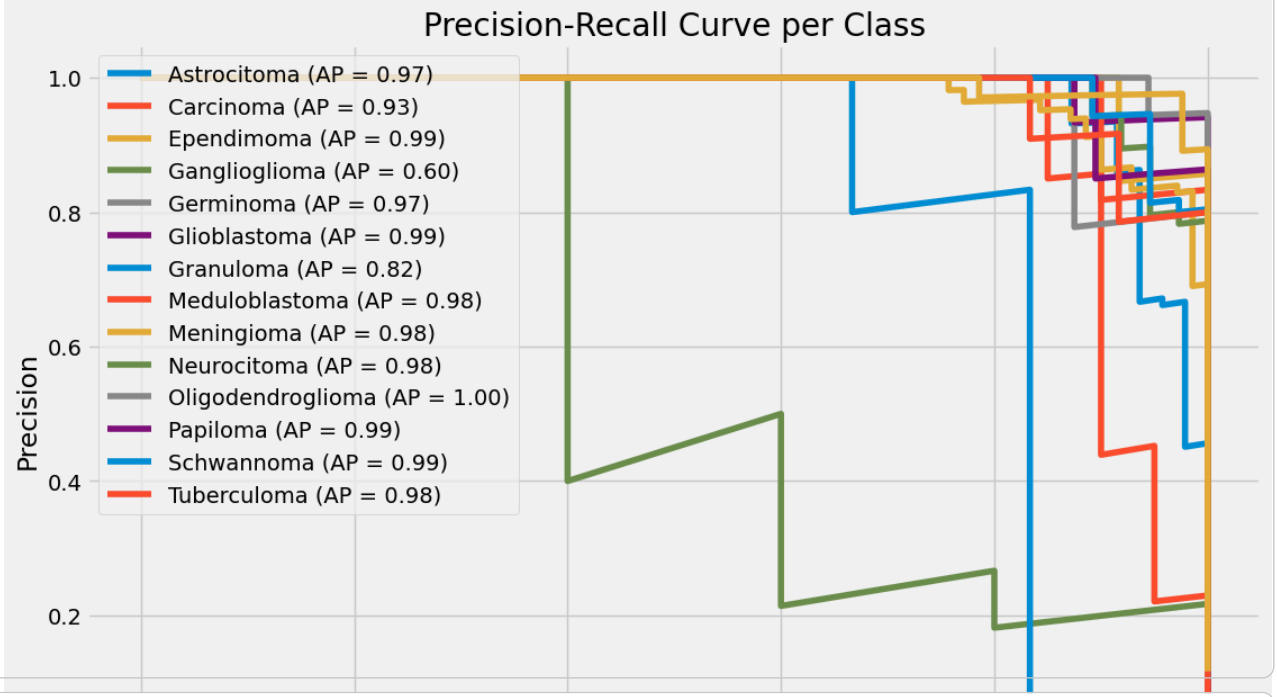
```
6/6 ━━━━━━━━━━━━━━━━━━━━ 15s 1s/step
Classification Report:

                  precision    recall  f1-score   support

      Astrocitoma       0.95      0.89      0.92        47
        Carcinoma       1.00      0.85      0.92        20
      Ependimoma       1.00      0.92      0.96        12
    Ganglioglioma       0.40      0.40      0.40         5
       Germinoma       0.70      0.88      0.78         8
     Glioblastoma       0.88      0.94      0.91        16
        Granuloma       0.71      0.83      0.77         6
   Meduloblastoma       0.83      1.00      0.91        10
       Meningioma       0.91      0.89      0.90        70
      Neurocitoma       0.94      0.89      0.92        37
Oligodendroglioma       0.90      1.00      0.95        18
         Papiloma       1.00      0.89      0.94        19
       Schwannoma       0.83      0.95      0.89        37
      Tuberculoma       0.92      0.92      0.92        12
          _NORMAL       0.98      0.98      0.98        42

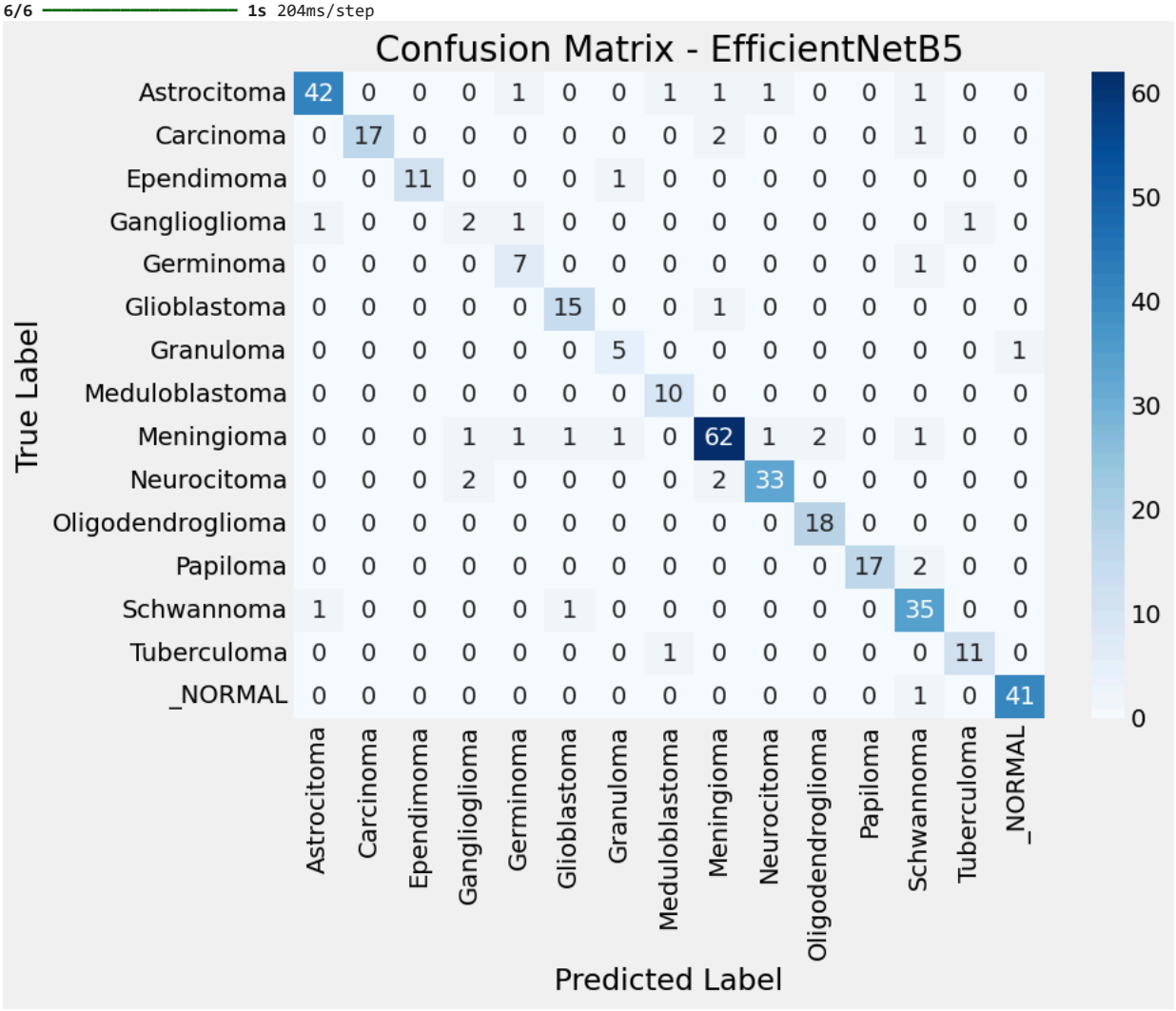
         accuracy                           0.91       359
        macro avg       0.86      0.88      0.87       359
     weighted avg       0.91      0.91      0.91       359

Macro Precision: 0.8644
Macro Recall: 0.8812
Macro F1 Score: 0.8700
```




```
# Plot confusion matrix for EfficientNetB5
plot_confusion_matrix(efficientNet_model_b0, test_gen, title="Confusion Matrix - EfficientNetB5")
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 1s 204ms/step
```




```
save_dir = '/content/drive/MyDrive/brain_tumor_project/models'
os.makedirs(save_dir, exist_ok=True)

# Save the 4 trained models
efficientNet_model_b0.save(f'{save_dir}/efficientNet_model_b0.keras')

history_dir = "/content/drive/MyDrive/brain_tumor_project/histories"
os.makedirs(history_dir, exist_ok=True)
pd.DataFrame(efficientnet_history.history).to_csv(
    f"{history_dir}/efficientnet_history.csv",
    index=False
)
```

Model 1B — EfficientNetB0 with a simplified head

This variation keeps the EfficientNetB0 backbone but simplifies the classification head.

Decision behind this version

A simpler head can reduce the number of trainable parameters in the task-specific layers. The goal is to test whether a lighter top architecture can preserve performance while improving efficiency or reducing overfitting.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adamax
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.callbacks import EarlyStopping

# Number of classes
class_count = len(train_gen.class_indices)

# Load pretrained EfficientNetB0 base model
base_model_Try = EfficientNetB0(
    include_top=False,
    weights='imagenet',
    input_shape=img_shape,
```

```
    pooling='avg'
)

# Freeze the base model so only the top classifier is trained first
base_model_Try.trainable = False

# Build the model
efficientNet_model_b0R = Sequential([
    base_model_Try,
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.2),
    Dense(class_count, activation='softmax')
])

# Compile the model
efficientNet_model_b0R.compile(
    optimizer=Adamax(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Show model summary
efficientNet_model_b0R.summary()

# Early stopping callback
early_stop = EarlyStopping(
    monitor='val_loss',          # watch validation loss
    patience=5,                  # stop after 5 epochs without improvement
    restore_best_weights=True,   # go back to the best model
    verbose=1
)

# Train the model
efficientnet_b0R_history = efficientNet_model_b0R.fit(
    train_gen,
    validation_data=valid_gen,
    epochs=30,
    callbacks=[early_stop],
    verbose=1
)
```

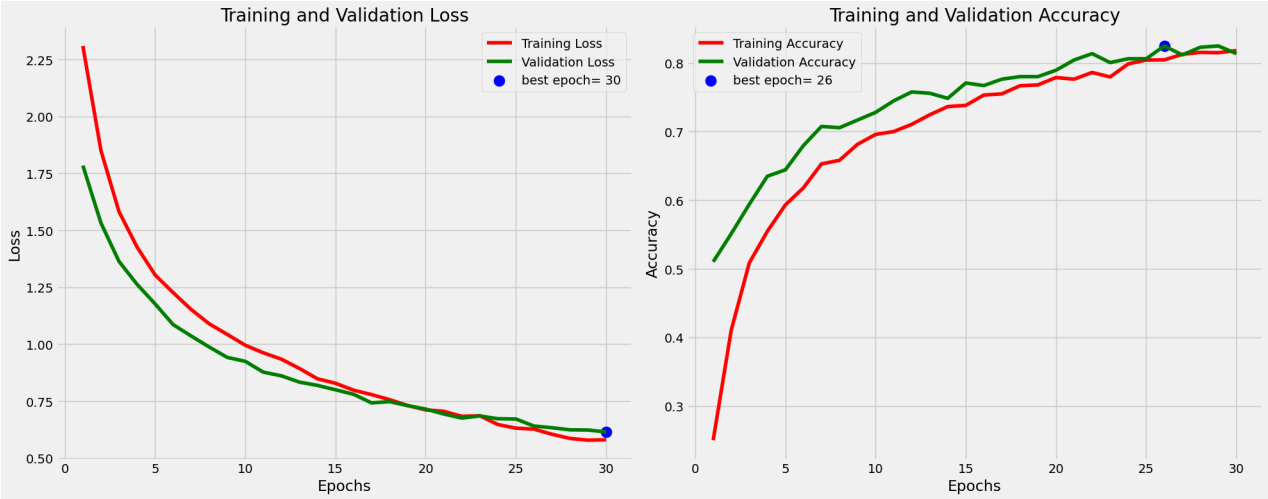
Model: "sequential_1"

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 1280)	4,049,571
dropout_2 (Dropout)	(None, 1280)	0
dense_2 (Dense)	(None, 128)	163,968
dropout_3 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 15)	1,935

Total params: 4,215,474 (16.08 MB)
Trainable params: 165,903 (648.06 KB)
Non-trainable params: 4,049,571 (15.45 MB)

```
Epoch 1/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 70s 678ms/step - accuracy: 0.2501 - loss: 2.3094 - val_accuracy: 0.5102 - val_loss: 1.7841
Epoch 2/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 17s 231ms/step - accuracy: 0.4105 - loss: 1.8507 - val_accuracy: 0.5512 - val_loss: 1.5324
Epoch 3/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 228ms/step - accuracy: 0.5085 - loss: 1.5825 - val_accuracy: 0.5940 - val_loss: 1.3636
Epoch 4/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 17s 231ms/step - accuracy: 0.5546 - loss: 1.4270 - val_accuracy: 0.6350 - val_loss: 1.2630
Epoch 5/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 17s 233ms/step - accuracy: 0.5932 - loss: 1.3044 - val_accuracy: 0.6443 - val_loss: 1.1772
Epoch 6/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 17s 233ms/step - accuracy: 0.6181 - loss: 1.2265 - val_accuracy: 0.6797 - val_loss: 1.0862
Epoch 7/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 227ms/step - accuracy: 0.6530 - loss: 1.1524 - val_accuracy: 0.7076 - val_loss: 1.0360
Epoch 8/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 225ms/step - accuracy: 0.6582 - loss: 1.0896 - val_accuracy: 0.7058 - val_loss: 0.9876
Epoch 9/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 219ms/step - accuracy: 0.6816 - loss: 1.0430 - val_accuracy: 0.7169 - val_loss: 0.9419
Epoch 10/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 226ms/step - accuracy: 0.6960 - loss: 0.9952 - val_accuracy: 0.7281 - val_loss: 0.9247
Epoch 11/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 15s 210ms/step - accuracy: 0.7001 - loss: 0.9620 - val_accuracy: 0.7449 - val_loss: 0.8772
Epoch 12/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 224ms/step - accuracy: 0.7106 - loss: 0.9343 - val_accuracy: 0.7579 - val_loss: 0.8611
Epoch 13/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 15s 208ms/step - accuracy: 0.7246 - loss: 0.8934 - val_accuracy: 0.7561 - val_loss: 0.8334
Epoch 14/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 17s 232ms/step - accuracy: 0.7366 - loss: 0.8480 - val_accuracy: 0.7486 - val_loss: 0.8191
Epoch 15/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 15s 211ms/step - accuracy: 0.7383 - loss: 0.8284 - val_accuracy: 0.7709 - val_loss: 0.7998
Epoch 16/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 222ms/step - accuracy: 0.7534 - loss: 0.7981 - val_accuracy: 0.7672 - val_loss: 0.7801
Epoch 17/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 219ms/step - accuracy: 0.7551 - loss: 0.7786 - val_accuracy: 0.7765 - val_loss: 0.7421
Epoch 18/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 218ms/step - accuracy: 0.7669 - loss: 0.7565 - val_accuracy: 0.7803 - val_loss: 0.7479
Epoch 19/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 221ms/step - accuracy: 0.7682 - loss: 0.7307 - val_accuracy: 0.7803 - val_loss: 0.7302
Epoch 20/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 220ms/step - accuracy: 0.7789 - loss: 0.7115 - val_accuracy: 0.7896 - val_loss: 0.7150
Epoch 21/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 15s 203ms/step - accuracy: 0.7765 - loss: 0.7053 - val_accuracy: 0.8045 - val_loss: 0.6935
Epoch 22/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 218ms/step - accuracy: 0.7863 - loss: 0.6832 - val_accuracy: 0.8138 - val_loss: 0.6754
Epoch 23/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 219ms/step - accuracy: 0.7798 - loss: 0.6857 - val_accuracy: 0.8007 - val_loss: 0.6850
Epoch 24/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 20s 209ms/step - accuracy: 0.7988 - loss: 0.6472 - val_accuracy: 0.8063 - val_loss: 0.6727
Epoch 25/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 225ms/step - accuracy: 0.8045 - loss: 0.6313 - val_accuracy: 0.8063 - val_loss: 0.6714
Epoch 26/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 215ms/step - accuracy: 0.8047 - loss: 0.6270 - val_accuracy: 0.8250 - val_loss: 0.6405
Epoch 27/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 221ms/step - accuracy: 0.8127 - loss: 0.6045 - val_accuracy: 0.8119 - val_loss: 0.6332
Epoch 28/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 15s 206ms/step - accuracy: 0.8156 - loss: 0.5862 - val_accuracy: 0.8231 - val_loss: 0.6239
Epoch 29/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 16s 217ms/step - accuracy: 0.8151 - loss: 0.5784 - val_accuracy: 0.8250 - val_loss: 0.6231
Epoch 30/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 15s 214ms/step - accuracy: 0.8182 - loss: 0.5805 - val_accuracy: 0.8138 - val_loss: 0.6145
Restoring model weights from the end of the best epoch: 30
```

```
plot_history(efficientnet_b0R_history)
```




```
ts_length = len(test_df)
test_batch_size = max(sorted([ts_length // n for n in range(1, ts_length + 1) if ts_length%n == 0 and ts_length/n <= 80]))
#test_steps = ts_length // test_batch_size

train_score = efficientNet_model_b0R.evaluate(train_gen,  verbose= 1)
valid_score = efficientNet_model_b0R.evaluate(valid_gen, verbose= 1)
test_score = efficientNet_model_b0R.evaluate(test_gen, verbose= 1)

print("Train Loss: ", train_score[0])
print("Train Accuracy: ", train_score[1])
print('-' * 20)
print("Validation Loss: ", valid_score[0])
print("Validation Accuracy: ", valid_score[1])
print('-' * 20)
print("Test Loss: ", test_score[0])
print("Test Accuracy: ", test_score[1])
```

```
72/72 ━━━━━━━━━━━━━━━━━━━━ 18s 244ms/step - accuracy: 0.9330 - loss: 0.2854
9/9 ━━━━━━━━━━━━━━━━━━━━ 2s 244ms/step - accuracy: 0.8138 - loss: 0.6145
6/6 ━━━━━━━━━━━━━━━━━━━━ 6s 1s/step - accuracy: 0.8162 - loss: 0.5950
Train Loss:  0.28543776273727417
Train Accuracy:  0.9329987168312073
--------------------
Validation Loss:  0.6144741773605347
Validation Accuracy:  0.8137802481651306
--------------------
Test Loss:  0.5950133800506592
Test Accuracy:  0.8161559700965881
```

```
evaluate_model_performance(efficientNet_model_b0R, test_gen)
```

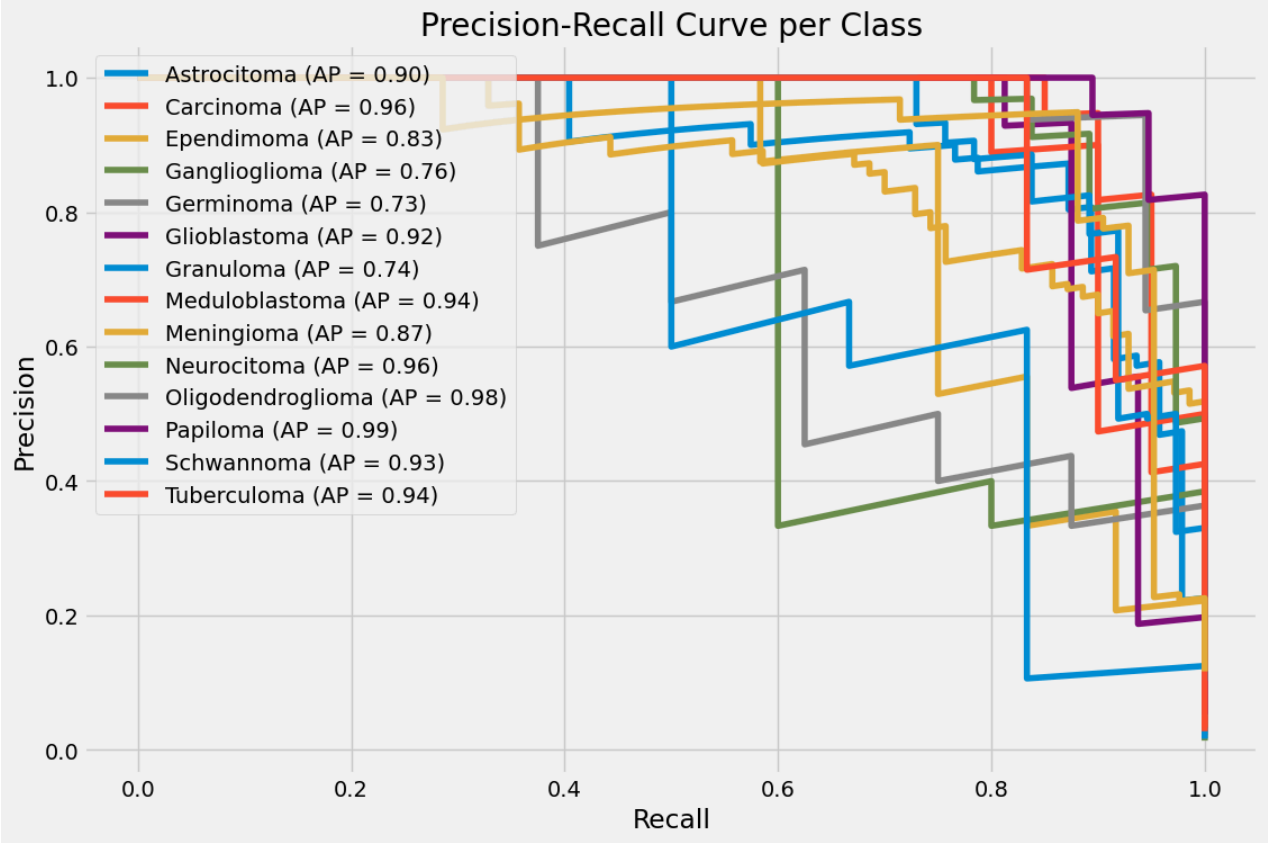
```
6/6 ━━━━━━━━━━━━━━━━━━━━ 15s 1s/step
Classification Report:

                  precision    recall  f1-score   support

      Astrocitoma       0.86      0.77      0.81        47
        Carcinoma       0.94      0.85      0.89        20
      Ependimoma        0.75      0.75      0.75        12
    Ganglioglioma       0.50      0.40      0.44         5
        Germinoma       0.45      0.62      0.53         8
     Glioblastoma       0.82      0.88      0.85        16
        Granuloma       1.00      0.50      0.67         6
   Meduloblastoma       0.82      0.90      0.86        10
       Meningioma       0.76      0.74      0.75        70
      Neurocitoma       0.87      0.89      0.88        37
Oligodendroglioma       0.85      0.94      0.89        18
         Papiloma       0.89      0.89      0.89        19
       Schwannoma       0.77      0.89      0.82        37
      Tuberculoma       1.00      0.67      0.80        12
          _NORMAL       0.84      0.90      0.87        42

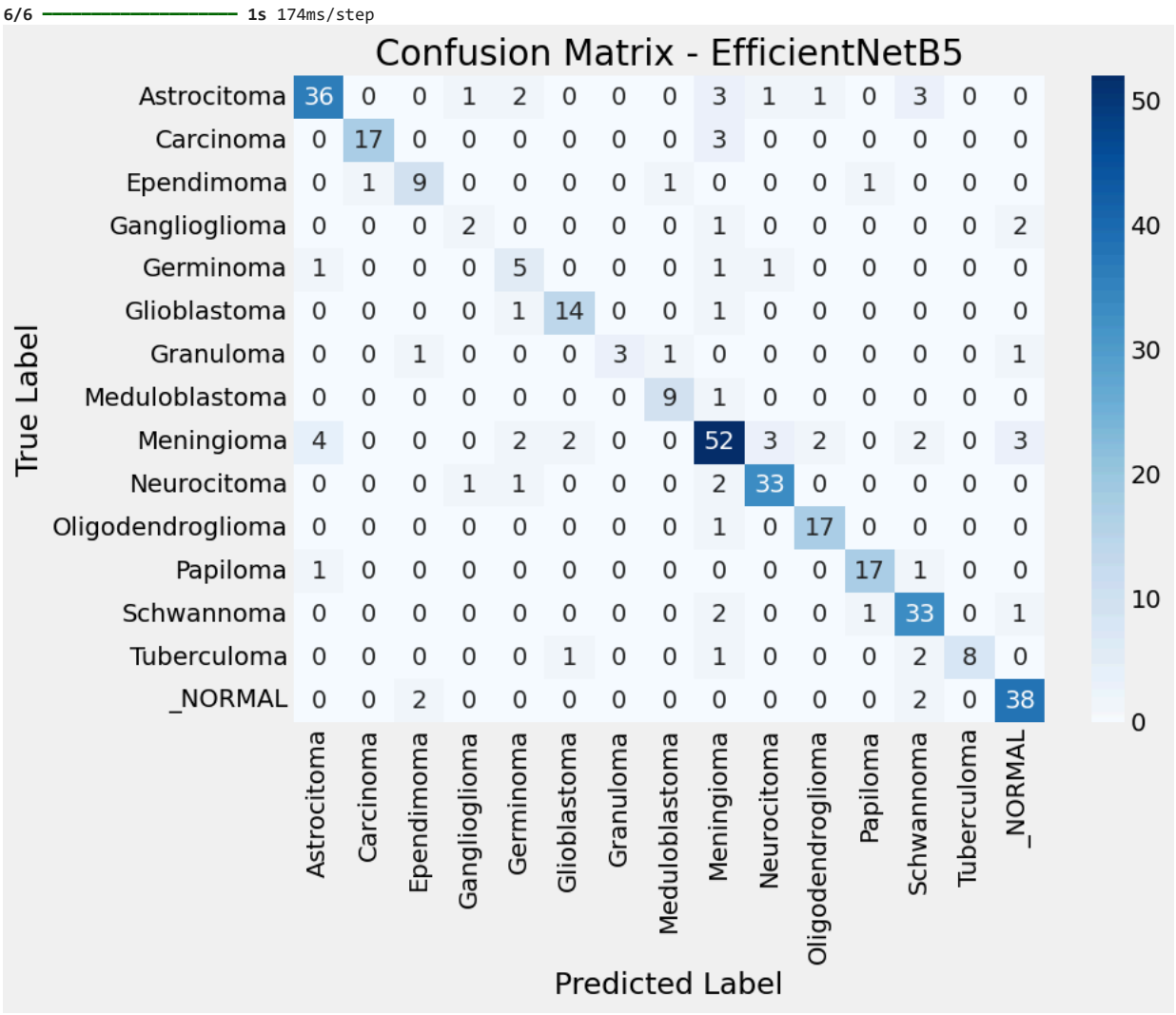
         accuracy                           0.82       359
        macro avg       0.81      0.77      0.78       359
     weighted avg       0.82      0.82      0.82       359

Macro Precision: 0.8092
Macro Recall: 0.7735
Macro F1 Score: 0.7812
```




```
# Plot confusion matrix for EfficientNetB5
plot_confusion_matrix(efficientNet_model_b0R, test_gen, title="Confusion Matrix - EfficientNetB5")
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 1s 174ms/step
```




```
save_dir = '/content/drive/MyDrive/brain_tumor_project/models'
os.makedirs(save_dir, exist_ok=True)

# Save the 4 trained models
efficientNet_model_b0R.save(f'{save_dir}/efficientNet_model_b0R.keras')

history_dir = "/content/drive/MyDrive/brain_tumor_project/histories"
os.makedirs(history_dir, exist_ok=True)

pd.DataFrame(efficientnet_b0R_history.history).to_csv(
    f"{history_dir}/efficientnet_b0R_history.csv",
    index=False
)
```

Model 2 — EfficientNetB5

This model scales up the EfficientNet family by replacing the B0 backbone with **EfficientNetB5**.

Why compare B5 against B0

The purpose of this comparison is to check whether a larger pretrained backbone produces a measurable accuracy gain that justifies its added complexity, training cost, and inference burden.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.optimizers import Adamax
from tensorflow.keras.applications import EfficientNetB5
from tensorflow.keras.regularizers import l2

class_count = len(train_gen.class_indices)

# Load the EfficientNetB5 base model
base_model_b5 = EfficientNetB5(
    include_top=False,
    weights='imagenet',
```

```
    input_shape=img_shape,
    pooling='max'
)

# Build the full model
efficientNet_model_b5 = Sequential([
    base_model_b5,
    Flatten(),
    Dropout(0.5),
    Dense(256, activation='relu', kernel_regularizer=l2(0.01)),
    Dropout(0.5),
    Dense(class_count, activation='softmax')
])

# Compile the model
efficientNet_model_b5.compile(
    optimizer=Adamax(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Show the model summary
efficientNet_model_b5.summary()
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb5_notop.h5
115263384/115263384 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Model: "sequential_2"
```

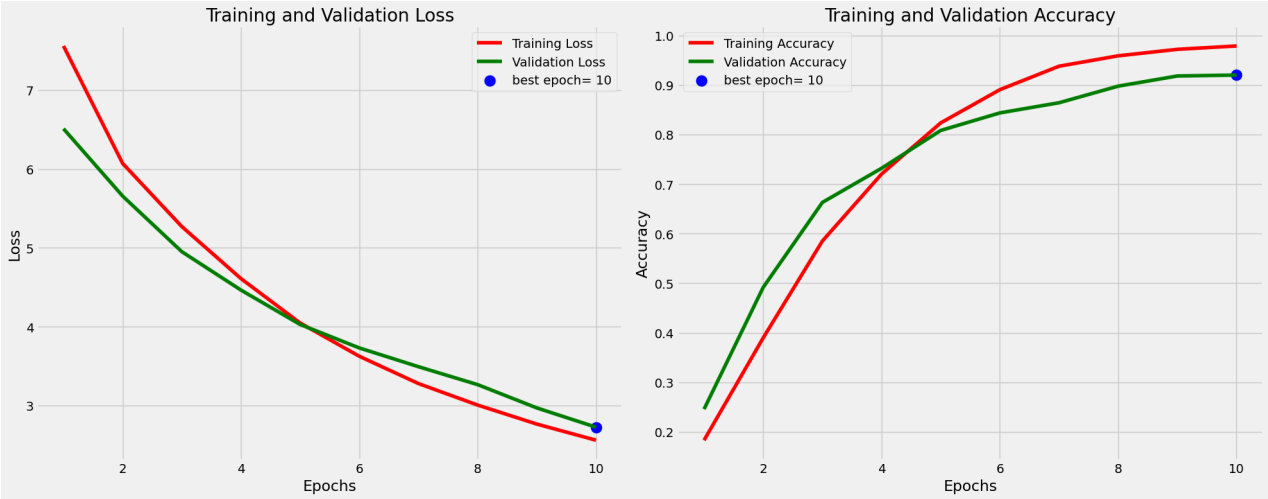
Layer (type)	Output Shape	Param #
efficientnetb5 (Functional)	(None, 2048)	28,513,527
flatten_1 (Flatten)	(None, 2048)	0
dropout_4 (Dropout)	(None, 2048)	0
dense_4 (Dense)	(None, 256)	524,544
dropout_5 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 15)	3,855

```
 Total params: 29,041,926 (110.79 MB)
 Trainable params: 28,869,183 (110.13 MB)
```

```
efficientnet_history_b5 = train_model(efficientNet_model_b5, train_gen, valid_gen, epochs=10, verbose=1)
```

```
Epoch 1/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 417s 3s/step - accuracy: 0.1831 - loss: 7.5611 - val_accuracy: 0.2458 - val_loss: 6.5114 - learni
Epoch 2/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 68s 943ms/step - accuracy: 0.3898 - loss: 6.0712 - val_accuracy: 0.4916 - val_loss: 5.6569 - lear
Epoch 3/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 72s 1000ms/step - accuracy: 0.5851 - loss: 5.2702 - val_accuracy: 0.6629 - val_loss: 4.9518 - lea
Epoch 4/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 76s 1s/step - accuracy: 0.7202 - loss: 4.6101 - val_accuracy: 0.7318 - val_loss: 4.4646 - learnir
Epoch 5/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.8234 - loss: 4.0469 - val_accuracy: 0.8082 - val_loss: 4.0284 - learnir
Epoch 6/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.8904 - loss: 3.6252 - val_accuracy: 0.8436 - val_loss: 3.7309 - learnir
Epoch 7/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.9378 - loss: 3.2789 - val_accuracy: 0.8641 - val_loss: 3.4928 - learnir
Epoch 8/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.9588 - loss: 3.0062 - val_accuracy: 0.8976 - val_loss: 3.2654 - learnir
Epoch 9/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 76s 1s/step - accuracy: 0.9718 - loss: 2.7653 - val_accuracy: 0.9181 - val_loss: 2.9700 - learnir
Epoch 10/10
72/72 ━━━━━━━━━━━━━━━━━━━━ 77s 1s/step - accuracy: 0.9786 - loss: 2.5608 - val_accuracy: 0.9199 - val_loss: 2.7260 - learnir
```

```
plot_history(efficientnet_history_b5)
```




```
ts_length = len(test_df)
test_batch_size = max(sorted([ts_length // n for n in range(1, ts_length + 1) if ts_length%n == 0 and ts_length/n <= 80]))
#test_steps = ts_length // test_batch_size

train_score = efficientNet_model_b5.evaluate(train_gen,  verbose= 1)
valid_score = efficientNet_model_b5.evaluate(valid_gen,  verbose= 1)
test_score = efficientNet_model_b5.evaluate(test_gen, verbose= 1)

print("Train Loss: ", train_score[0])
print("Train Accuracy: ", train_score[1])
print('-' * 20)
print("Validation Loss: ", valid_score[0])
print("Validation Accuracy: ", valid_score[1])
print('-' * 20)
print("Test Loss: ", test_score[0])
print("Test Accuracy: ", test_score[1])
```

```
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 398ms/step - accuracy: 0.9972 - loss: 2.4015
9/9 ━━━━━━━━━━━━━━━━━━━━ 2s 243ms/step - accuracy: 0.9199 - loss: 2.7260
6/6 ━━━━━━━━━━━━━━━━━━━━ 23s 4s/step - accuracy: 0.9276 - loss: 2.6670
Train Loss:  2.4015119075775146
Train Accuracy:  0.9971628189086914
--------------------
Validation Loss:  2.7260453701019287
Validation Accuracy:  0.9199255108833313
--------------------
Test Loss:  2.6670033931732178
Test Accuracy:  0.9275766015052795
```

```
evaluate_model_performance(efficientNet_model_b5, test_gen)
```

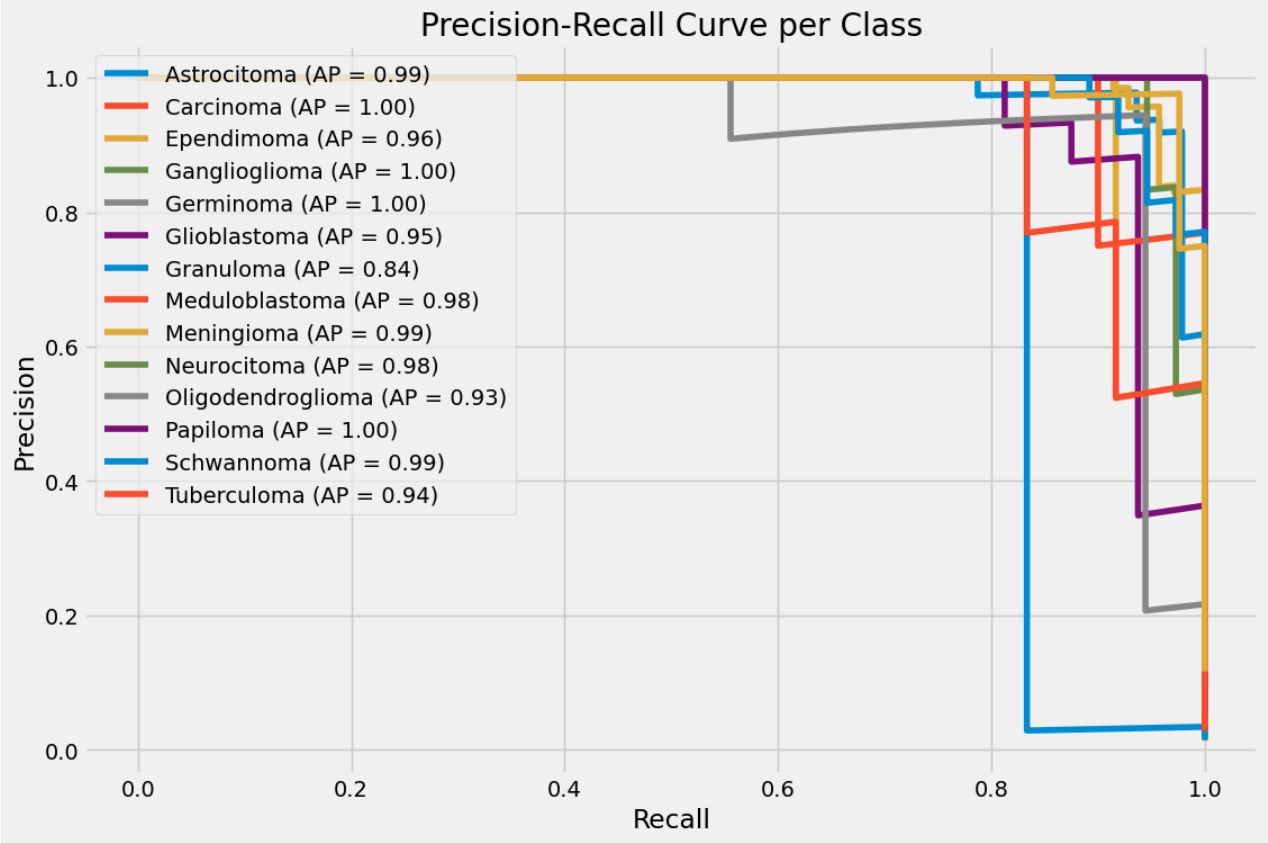
```
6/6 ━━━━━━━━━━━━━━━━━━━━ 32s 3s/step
Classification Report:

                  precision    recall  f1-score   support

      Astrocitoma       0.98      0.91      0.95        47
        Carcinoma       1.00      1.00      1.00        20
       Ependimoma       1.00      0.92      0.96        12
    Ganglioglioma       1.00      0.80      0.89         5
        Germinoma       1.00      1.00      1.00         8
     Glioblastoma       0.79      0.94      0.86        16
        Granuloma       1.00      0.67      0.80         6
   Meduloblastoma       0.77      1.00      0.87        10
       Meningioma       0.96      0.91      0.93        70
      Neurocitoma       0.90      0.95      0.92        37
Oligodendroglioma       0.94      0.83      0.88        18
         Papiloma       1.00      1.00      1.00        19
       Schwannoma       0.85      0.95      0.90        37
      Tuberculoma       1.00      0.75      0.86        12
          _NORMAL       0.91      0.98      0.94        42

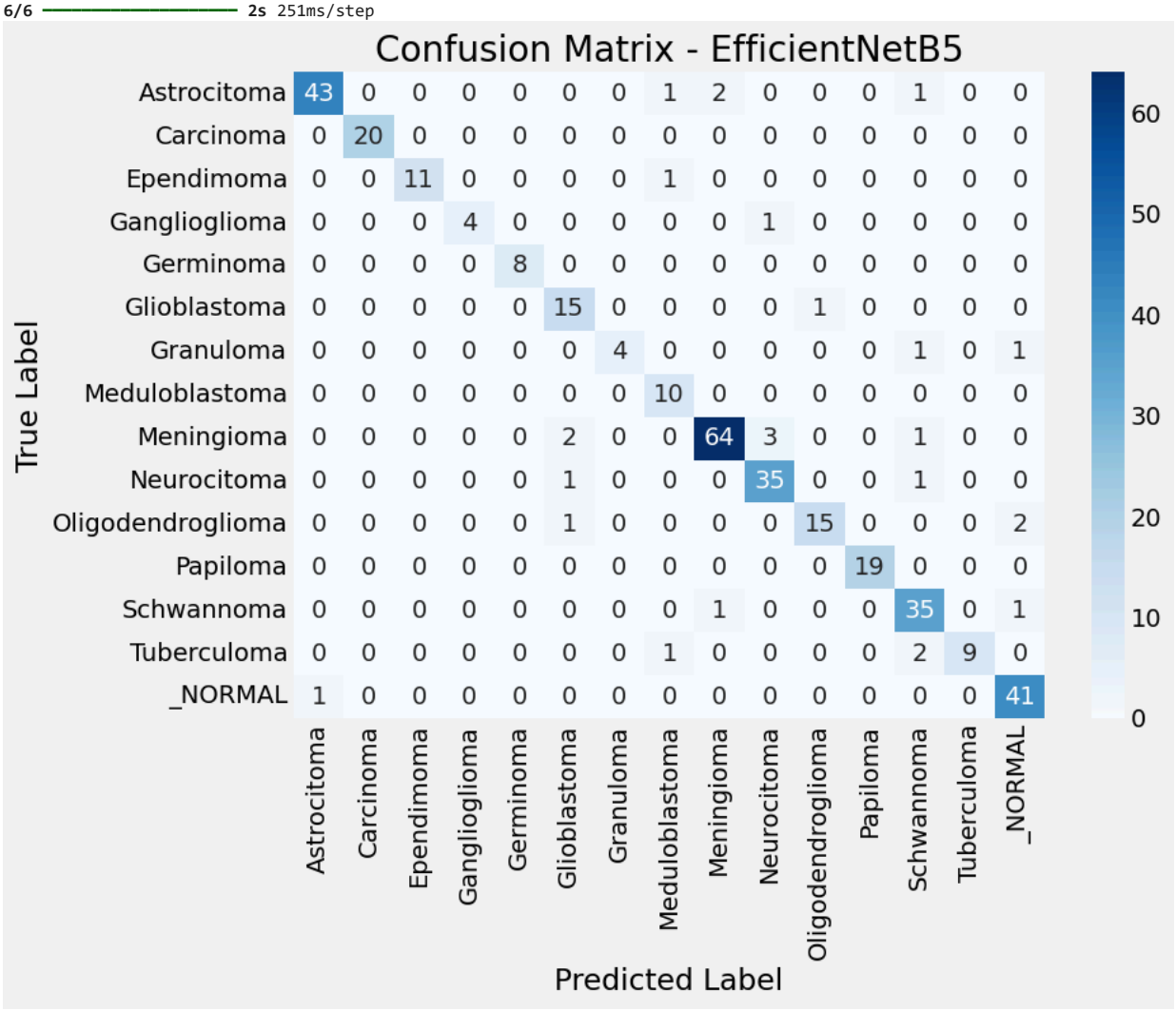
         accuracy                           0.93       359
        macro avg       0.94      0.91      0.92       359
     weighted avg       0.93      0.93      0.93       359

Macro Precision: 0.9394
Macro Recall: 0.9068
Macro F1 Score: 0.9168
```




```
# Plot confusion matrix for EfficientNetB5
plot_confusion_matrix(efficientNet_model_b5, test_gen, title="Confusion Matrix - EfficientNetB5")
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 2s 251ms/step
```




```
save_dir = '/content/drive/MyDrive/brain_tumor_project/models'
os.makedirs(save_dir, exist_ok=True)

# Save the 4 trained models
efficientNet_model_b5.save(f'{save_dir}/efficientNet_model_b5.keras')

history_dir = "/content/drive/MyDrive/brain_tumor_project/histories"
os.makedirs(history_dir, exist_ok=True)

pd.DataFrame(efficientnet_history_b5.history).to_csv(
    f"{history_dir}/efficientnet_history_b5.csv",
    index=False
)
```

Model 3 — DenseNet121

This model introduces **DenseNet121** as an alternative transfer-learning backbone.

Research motivation

> DenseNets "encourage feature reuse, and substantially reduce the number of parameters."
> — Huang et al., *Densely Connected Convolutional Networks* (2017)

Why DenseNet121 is included

DenseNet121 is useful in this comparison because its dense connectivity differs from EfficientNet's scaling philosophy. Including it gives the project a broader architectural comparison instead of evaluating only one model family.

```
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.optimizers import Adamax
from tensorflow.keras.regularizers import l2

# Load the DenseNet121 base model
```

```
base_model_d121 = DenseNet121(
    include_top=False,
    weights='imagenet',
    input_shape=img_shape,
    pooling='max'  # Global max pooling
)

class_count = len(train_gen.class_indices)

# Build the full model
densenet121_model = Sequential([
    base_model_d121,
    Flatten(),
    Dropout(0.5),
    Dense(256, activation='relu', kernel_regularizer=l2(0.01)),
    Dropout(0.5),
    Dense(class_count, activation='softmax')  # Output layer
])

# Compile the model
densenet121_model.compile(
    optimizer=Adamax(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Model summary
densenet121_model.summary()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet121_weights_tf_dim_order
29084464/29084464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
densenet121 (Functional)	(None, 1024)	7,037,504
flatten_2 (Flatten)	(None, 1024)	0
dropout_6 (Dropout)	(None, 1024)	0
dense_6 (Dense)	(None, 256)	262,400
dropout_7 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 15)	3,855

```
 Total params: 7,303,759 (27.86 MB)
 Trainable params: 7,220,111 (27.54 MB)
```

```
DenseNet_history = train_model(densenet121_model, train_gen, valid_gen, epochs=20, verbose=1)
```

```
Epoch 1/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 419s 3s/step - accuracy: 0.1630 - loss: 7.2453 - val_accuracy: 0.1676 - val_loss: 4.9747 - learni
Epoch 2/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 47s 644ms/step - accuracy: 0.2593 - loss: 4.6063 - val_accuracy: 0.3017 - val_loss: 4.2732 - lear
Epoch 3/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 45s 630ms/step - accuracy: 0.3426 - loss: 4.1604 - val_accuracy: 0.5307 - val_loss: 3.7442 - lear
Epoch 4/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 83s 650ms/step - accuracy: 0.4101 - loss: 3.8433 - val_accuracy: 0.6220 - val_loss: 3.2808 - lear
Epoch 5/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 45s 622ms/step - accuracy: 0.4980 - loss: 3.4723 - val_accuracy: 0.6629 - val_loss: 3.0478 - lear
Epoch 6/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 634ms/step - accuracy: 0.5766 - loss: 3.1642 - val_accuracy: 0.6834 - val_loss: 2.8984 - lear
Epoch 7/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 639ms/step - accuracy: 0.6401 - loss: 2.9110 - val_accuracy: 0.7393 - val_loss: 2.6009 - lear
Epoch 8/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 634ms/step - accuracy: 0.6966 - loss: 2.6943 - val_accuracy: 0.7244 - val_loss: 2.6769 - lear
Epoch 9/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 45s 625ms/step - accuracy: 0.7680 - loss: 2.4026 - val_accuracy: 0.7207 - val_loss: 2.7161 - lear
Epoch 10/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 45s 627ms/step - accuracy: 0.8383 - loss: 2.1535 - val_accuracy: 0.8696 - val_loss: 2.0643 - lear
Epoch 11/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 634ms/step - accuracy: 0.8756 - loss: 2.0007 - val_accuracy: 0.8212 - val_loss: 2.1170 - lear
Epoch 12/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 47s 645ms/step - accuracy: 0.8987 - loss: 1.8961 - val_accuracy: 0.8939 - val_loss: 1.9183 - lear
Epoch 13/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 632ms/step - accuracy: 0.9273 - loss: 1.7833 - val_accuracy: 0.8715 - val_loss: 1.9588 - lear
Epoch 14/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 631ms/step - accuracy: 0.9356 - loss: 1.7158 - val_accuracy: 0.8585 - val_loss: 1.9367 - lear
Epoch 15/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 634ms/step - accuracy: 0.9555 - loss: 1.6243 - val_accuracy: 0.9218 - val_loss: 1.7291 - lear
Epoch 16/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 640ms/step - accuracy: 0.9644 - loss: 1.5809 - val_accuracy: 0.9162 - val_loss: 1.7610 - lear
Epoch 17/20
```

```
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 630ms/step - accuracy: 0.9712 - loss: 1.5432 - val_accuracy: 0.8994 - val_loss: 1.7503 - lear
Epoch 18/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 45s 629ms/step - accuracy: 0.9838 - loss: 1.4951 - val_accuracy: 0.9348 - val_loss: 1.6384 - lear
Epoch 19/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 631ms/step - accuracy: 0.9806 - loss: 1.4894 - val_accuracy: 0.9330 - val_loss: 1.6520 - lear
Epoch 20/20
72/72 ━━━━━━━━━━━━━━━━━━━━ 46s 635ms/step - accuracy: 0.9858 - loss: 1.4681 - val_accuracy: 0.9441 - val_loss: 1.6565 - lear
```

```
plot_history(DenseNet_history)
```




```
evaluate_model_performance(densenet121_model, test_gen)
```

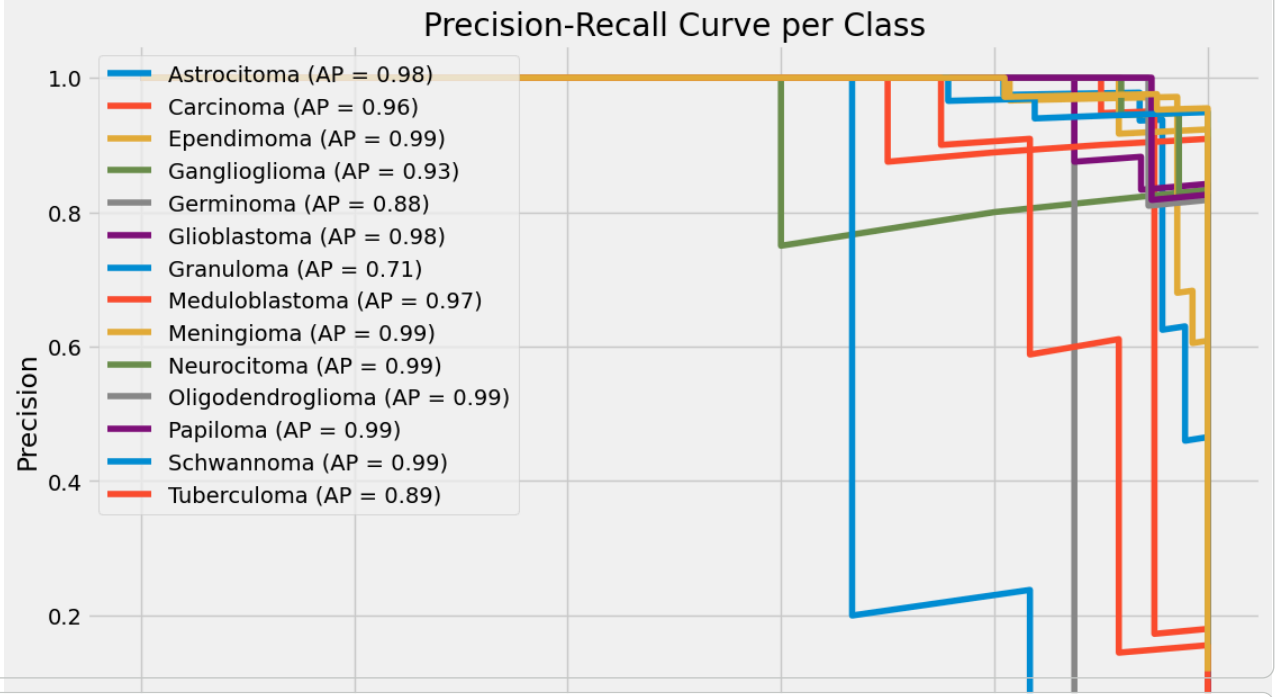
```
6/6 ━━━━━━━━━━━━━━━━━━━━ 32s 4s/step
Classification Report:

                  precision    recall  f1-score   support

      Astrocitoma       0.94      0.96      0.95        47
        Carcinoma       1.00      0.90      0.95        20
       Ependimoma       0.92      0.92      0.92        12
    Ganglioglioma       0.75      0.60      0.67         5
        Germinoma       0.78      0.88      0.82         8
     Glioblastoma       0.88      0.94      0.91        16
        Granuloma       1.00      0.67      0.80         6
   Meduloblastoma       0.83      1.00      0.91        10
       Meningioma       0.97      0.93      0.95        70
      Neurocitoma       0.97      0.97      0.97        37
Oligodendroglioma       1.00      0.94      0.97        18
         Papiloma       0.95      0.95      0.95        19
       Schwannoma       0.95      1.00      0.97        37
      Tuberculoma       0.75      0.75      0.75        12
          _NORMAL       0.95      1.00      0.98        42

         accuracy                           0.94       359
        macro avg       0.91      0.89      0.90       359
     weighted avg       0.94      0.94      0.94       359

Macro Precision: 0.9094
Macro Recall: 0.8931
Macro F1 Score: 0.8974
```




```
ts_length = len(test_df)
test_batch_size = max(sorted([ts_length // n for n in range(1, ts_length + 1) if ts_length%n == 0 and ts_length/n <= 80]))
test_steps = ts_length // test_batch_size

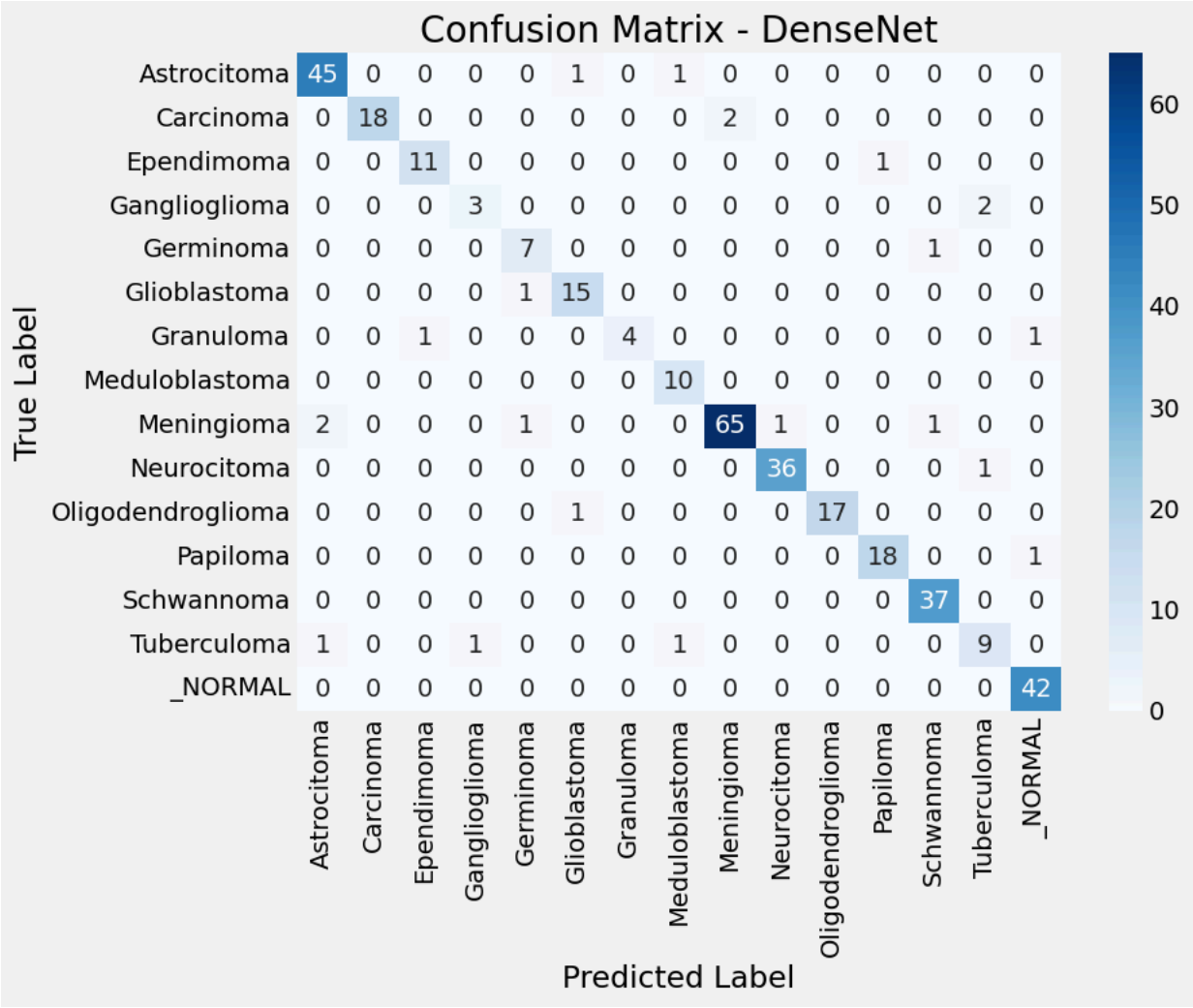
train_score = densenet121_model.evaluate(train_gen, steps= test_steps, verbose= 1)
valid_score = densenet121_model.evaluate(valid_gen, steps= test_steps, verbose= 1)
test_score = densenet121_model.evaluate(test_gen, steps= test_steps, verbose= 1)

print("Train Loss: ", train_score[0])
print("Train Accuracy: ", train_score[1])
print('-' * 20)
print("Validation Loss: ", valid_score[0])
print("Validation Accuracy: ", valid_score[1])
print('-' * 20)
print("Test Loss: ", test_score[0])
print("Test Accuracy: ", test_score[1])
```

```
359/359 ━━━━━━━━━━━━━━━━━━━━ 25s 68ms/step - accuracy: 0.9987 - loss: 1.4395
359/359 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9348 - loss: 1.6384
359/359 ━━━━━━━━━━━━━━━━━━━━ 12s 31ms/step - accuracy: 0.9387 - loss: 1.7715
Train Loss:  1.4394582509994507
Train Accuracy:  0.9986905455589294
--------------------
Validation Loss:  1.6384122371673584
Validation Accuracy:  0.9348230957984924
--------------------
Test Loss:  1.7714940309524536
Test Accuracy:  0.9387186765670776
```

```
# Plot confusion matrix for ResNet50
plot_confusion_matrix(densenet121_model, test_gen, title="Confusion Matrix - DenseNet")
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 1s 195ms/step
```




```
import os
import pandas as pd

save_dir = "/content/drive/MyDrive/brain_tumor_project/models"
history_dir = "/content/drive/MyDrive/brain_tumor_project/histories"

os.makedirs(save_dir, exist_ok=True)
os.makedirs(history_dir, exist_ok=True)

# Save history FIRST
pd.DataFrame(DenseNet_history.history).to_csv(
    f"{history_dir}/DenseNet_history.csv",
    index=False
)

print("DenseNet history saved successfully.")

# Then save model
densenet121_model.save(f"{save_dir}/densenet121_model.keras")

print("DenseNet model saved successfully.")
```

```
DenseNet history saved successfully.
DenseNet model saved successfully.
```

Model 4 - Custom Model

```
# ── MODEL 0: Custom CNN trained from scratch ──────────────────────────
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv2D, BatchNormalization, Activation,
    MaxPooling2D, GlobalAveragePooling2D,
    Dense, Dropout
)
from tensorflow.keras.optimizers import Adamax
```

```
from tensorflow.keras.optimizers import Adamax

class_count = len(train_gen.class_indices)

def conv_block(model, filters, dropout_rate=0.0):
    """Two Conv → BN → ReLU layers followed by MaxPool and optional Dropout."""
    model.add(Conv2D(filters, (3, 3), padding='same', use_bias=False))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(Conv2D(filters, (3, 3), padding='same', use_bias=False))
    model.add(BatchNormalization())
    model.add(Activation('relu'))

    model.add(MaxPooling2D(pool_size=(2, 2)))
    if dropout_rate > 0:
        model.add(Dropout(dropout_rate))

custom_cnn = Sequential(name="Custom_CNN")

# Input: 224×224×3
conv_block(custom_cnn, filters=32,  dropout_rate=0.1)   # → 112×112×32
conv_block(custom_cnn, filters=64,  dropout_rate=0.2)   # → 56×56×64
conv_block(custom_cnn, filters=128, dropout_rate=0.3)   # → 28×28×128
conv_block(custom_cnn, filters=256, dropout_rate=0.4)   # → 14×14×256
conv_block(custom_cnn, filters=512, dropout_rate=0.4)   # → 7×7×512

# Head
custom_cnn.add(GlobalAveragePooling2D())         # → 512
custom_cnn.add(Dense(256, use_bias=False))
custom_cnn.add(BatchNormalization())
custom_cnn.add(Activation('relu'))
custom_cnn.add(Dropout(0.5))
custom_cnn.add(Dense(class_count, activation='softmax'))

custom_cnn.build(input_shape=(None, 224, 224, 3))

custom_cnn.compile(
    optimizer=Adamax(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

custom_cnn.summary()
```

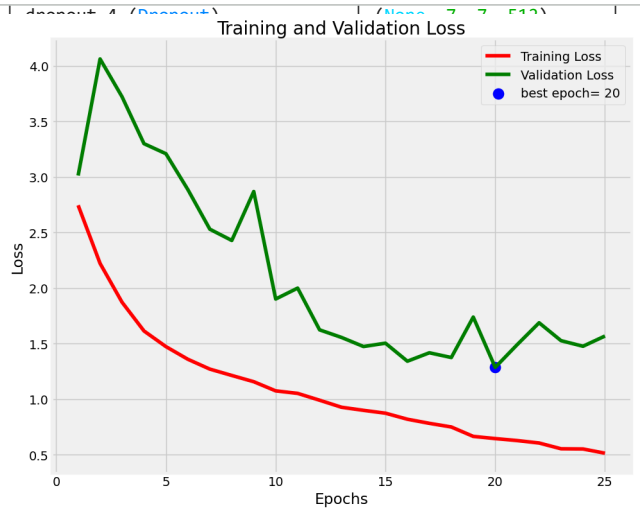
Model: "Custom_CNN"

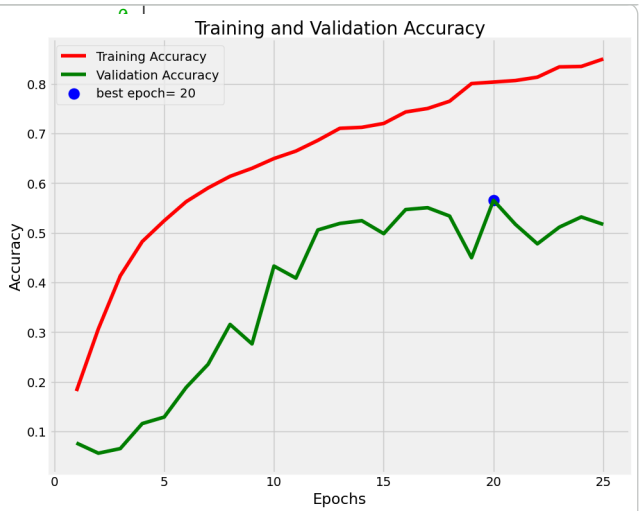
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	864
batch_normalization (BatchNormalization)	(None, 224, 224, 32)	128
activation (Activation)	(None, 224, 224, 32)	0
conv2d_1 (Conv2D)	(None, 224, 224, 32)	9,216
activation_1 (Activation)	(None, 224, 224, 32)	0
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
dropout (Dropout)	(None, 112, 112, 32)	0
conv2d_2 (Conv2D)	(None, 112, 112, 64)	18,432
batch_normalization_2 (BatchNormalization)	(None, 112, 112, 64)	256
activation_2 (Activation)	(None, 112, 112, 64)	0
conv2d_3 (Conv2D)	(None, 112, 112, 64)	36,864
batch_normalization_3 (BatchNormalization)	(None, 112, 112, 64)	256
activation_3 (Activation)	(None, 112, 112, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
dropout_1 (Dropout)	(None, 56, 56, 64)	0
conv2d_4 (Conv2D)	(None, 56, 56, 128)	73,728
batch_normalization_4 (BatchNormalization)	(None, 56, 56, 128)	512
activation_4 (Activation)	(None, 56, 56, 128)	0
conv2d_5 (Conv2D)	(None, 56, 56, 128)	147,456
batch_normalization_5 (BatchNormalization)	(None, 56, 56, 128)	512
activation_5 (Activation)	(None, 56, 56, 128)	0
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
dropout_2 (Dropout)	(None, 28, 28, 128)	0
conv2d_6 (Conv2D)	(None, 28, 28, 256)	294,912
batch_normalization_6 (BatchNormalization)	(None, 28, 28, 256)	1,024
activation_6 (Activation)	(None, 28, 28, 256)	0
conv2d_7 (Conv2D)	(None, 28, 28, 256)	589,824
batch_normalization_7 (BatchNormalization)	(None, 28, 28, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 256)	0
dropout_3 (Dropout)	(None, 14, 14, 256)	0
conv2d_8 (Conv2D)	(None, 14, 14, 512)	1,179,648
batch_normalization_8 (BatchNormalization)	(None, 14, 14, 512)	2,048
activation_8 (Activation)	(None, 14, 14, 512)	0
conv2d_9 (Conv2D)	(None, 14, 14, 512)	2,359,296
batch_normalization_9 (BatchNormalization)	(None, 14, 14, 512)	2,048
activation_9 (Activation)	(None, 14, 14, 512)	0
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 512)	0

```
# Training — stessa pipeline degli altri modelli
custom_cnn_history = train_model(custom_cnn, train_gen, valid_gen, epochs=30, verbose=1)
```

```
Epoch 1/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 121s 970ms/step - accuracy: 0.1811 - loss: 2.7445 - val_accuracy: 0.0765 - val_loss: 3.0119 - lea
Epoch 2/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 401ms/step - accuracy: 0.3063 - loss: 2.2208 - val_accuracy: 0.0560 - val_loss: 4.0611 - lear
Epoch 3/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 30s 410ms/step - accuracy: 0.4133 - loss: 1.8718 - val_accuracy: 0.0653 - val_loss: 3.7212 - lear
Epoch 4/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 28s 391ms/step - accuracy: 0.4822 - loss: 1.6130 - val_accuracy: 0.1157 - val_loss: 3.2988 - lear
Epoch 5/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 397ms/step - accuracy: 0.5241 - loss: 1.4721 - val_accuracy: 0.1287 - val_loss: 3.2083 - lear
Epoch 6/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 396ms/step - accuracy: 0.5625 - loss: 1.3582 - val_accuracy: 0.1884 - val_loss: 2.8851 - lear
Epoch 7/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 396ms/step - accuracy: 0.5902 - loss: 1.2696 - val_accuracy: 0.2351 - val_loss: 2.5290 - lear
Epoch 8/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 28s 393ms/step - accuracy: 0.6136 - loss: 1.2125 - val_accuracy: 0.3153 - val_loss: 2.4275 - lear
Epoch 9/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 395ms/step - accuracy: 0.6297 - loss: 1.1562 - val_accuracy: 0.2761 - val_loss: 2.8681 - lear
Epoch 10/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 395ms/step - accuracy: 0.6494 - loss: 1.0738 - val_accuracy: 0.4328 - val_loss: 1.9005 - lear
Epoch 11/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 396ms/step - accuracy: 0.6644 - loss: 1.0512 - val_accuracy: 0.4086 - val_loss: 1.9987 - lear
Epoch 12/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 394ms/step - accuracy: 0.6860 - loss: 0.9896 - val_accuracy: 0.5056 - val_loss: 1.6230 - lear
Epoch 13/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 28s 393ms/step - accuracy: 0.7102 - loss: 0.9263 - val_accuracy: 0.5187 - val_loss: 1.5546 - lear
Epoch 14/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 394ms/step - accuracy: 0.7122 - loss: 0.8985 - val_accuracy: 0.5243 - val_loss: 1.4729 - lear
Epoch 15/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 394ms/step - accuracy: 0.7201 - loss: 0.8729 - val_accuracy: 0.4981 - val_loss: 1.5030 - lear
Epoch 16/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 394ms/step - accuracy: 0.7432 - loss: 0.8185 - val_accuracy: 0.5466 - val_loss: 1.3407 - lear
Epoch 17/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 394ms/step - accuracy: 0.7502 - loss: 0.7815 - val_accuracy: 0.5504 - val_loss: 1.4166 - lear
Epoch 18/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 28s 393ms/step - accuracy: 0.7648 - loss: 0.7488 - val_accuracy: 0.5336 - val_loss: 1.3739 - lear
Epoch 19/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 28s 393ms/step - accuracy: 0.8003 - loss: 0.6640 - val_accuracy: 0.4496 - val_loss: 1.7380 - lear
Epoch 20/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 394ms/step - accuracy: 0.8034 - loss: 0.6442 - val_accuracy: 0.5653 - val_loss: 1.2854 - lear
Epoch 21/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 395ms/step - accuracy: 0.8065 - loss: 0.6265 - val_accuracy: 0.5168 - val_loss: 1.4894 - lear
Epoch 22/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 395ms/step - accuracy: 0.8132 - loss: 0.6044 - val_accuracy: 0.4776 - val_loss: 1.6866 - lear
Epoch 23/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 395ms/step - accuracy: 0.8340 - loss: 0.5525 - val_accuracy: 0.5112 - val_loss: 1.5254 - lear
Epoch 24/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 395ms/step - accuracy: 0.8348 - loss: 0.5507 - val_accuracy: 0.5317 - val_loss: 1.4753 - lear
Epoch 25/30
72/72 ━━━━━━━━━━━━━━━━━━━━ 29s 395ms/step - accuracy: 0.8499 - loss: 0.5127 - val_accuracy: 0.5168 - val_loss: 1.5675 - lear
```

```
plot_history(custom_cnn_history)
```






```
# Valutazione
ts_length = len(test_df)
test_batch_size = max(sorted([ts_length // n for n in range(1, ts_length + 1)
                              if ts_length % n == 0 and ts_length / n <= 80]))
test_steps = ts_length // test_batch_size

train_score = custom_cnn.evaluate(train_gen, steps=test_steps, verbose=1)
valid_score = custom_cnn.evaluate(valid_gen, steps=test_steps, verbose=1)
test_score  = custom_cnn.evaluate(test_gen,  steps=test_steps, verbose=1)

print(f"Train  — Loss: {train_score[0]:.4f}  Acc: {train_score[1]:.4f}")
print(f"Valid  — Loss: {valid_score[0]:.4f}  Acc: {valid_score[1]:.4f}")
print(f"Test   — Loss: {test_score[0]:.4f}  Acc: {test_score[1]:.4f}")
```

```
359/359 ━━━━━━━━━━━━━━━━━━━━ 17s 45ms/step - accuracy: 0.6472 - loss: 1.1165
359/359 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.5653 - loss: 1.2854
359/359 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.5627 - loss: 1.4741
Train  — Loss: 1.1165  Acc: 0.6472
Valid  — Loss: 1.2854  Acc: 0.5653
Test   — Loss: 1.4741  Acc: 0.5627
```

```
evaluate_model_performance(custom_cnn, test_gen)
plot_confusion_matrix(custom_cnn, test_gen, title="Confusion Matrix - Custom CNN")
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 3s 367ms/step
Classification Report:

                   precision    recall  f1-score   support

      Astrocitoma       0.64      0.45      0.53        47
        Carcinoma       0.65      0.85      0.74        20
       Ependimoma       0.62      0.42      0.50        12
    Ganglioglioma       0.14      1.00      0.24         5
        Germinoma       0.75      0.75      0.75         8
     Glioblastoma       0.71      0.31      0.43        16
        Granuloma       0.17      0.67      0.27         6
   Meduloblastoma       0.27      0.90      0.42        10
       Meningioma       0.63      0.71      0.67        70
      Neurocitoma       1.00      0.16      0.28        37
Oligodendroglioma       0.87      0.72      0.79        18
         Papiloma       0.74      0.74      0.74        19
       Schwannoma       0.69      0.49      0.57        37
      Tuberculoma       0.46      0.50      0.48        12
          _NORMAL       0.92      0.55      0.69        42

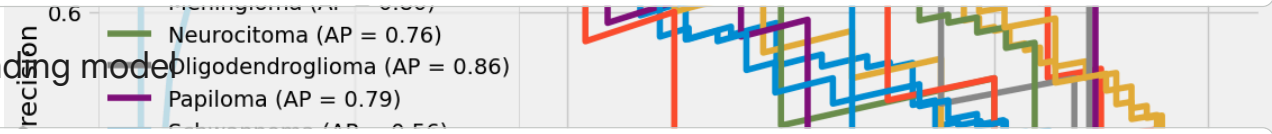
         accuracy                           0.56       359
        macro avg       0.62      0.61      0.54       359
     weighted avg       0.70      0.56      0.58       359

Macro Precision: 0.6176
Macro Recall: 0.6142
```

```
import os

save_dir = '/content/drive/MyDrive/brain_tumor_project/models'
os.makedirs(save_dir, exist_ok=True)
custom_cnn.save(f'{save_dir}/custom_cnn.keras')

history_dir = "/content/drive/MyDrive/brain_tumor_project/histories"
os.makedirs(history_dir, exist_ok=True)
pd.DataFrame(custom_cnn_history.history).to_csv(
    f"{history_dir}/custom_cnn_history.csv",
    index=False
)
```



loading model

```
# Load saved models and training histories

from google.colab import drive
from tensorflow.keras.models import load_model
import os
import pandas as pd
import json
import pickle

drive.mount('/content/drive')

base_path = '/content/drive/MyDrive'

# ------------------------------
# 1) LOAD SAVED MODELS
# ------------------------------

model_files = []

for root, dirs, files in os.walk(base_path):
    for file in files:
        fname = file.lower()

        # Load only real model files, not history files
        if (fname.endswith('.keras') or fname.endswith('.h5')) and 'history' not in fname:
            model_files.append(os.path.join(root, file))

print("Found model files:")
for f in model_files:
    print(f)

for f in model_files:
    fname = os.path.basename(f).lower()

    if 'b0r' in fname or 'efficientnet_model_b0r' in fname:
        efficientNet_model_b0R = load_model(f)
        print(f"Loaded efficientNet_model_b0R from {f}")
```

```
    elif 'b0' in fname and 'b0r' not in fname:
        efficientNet_model_b0 = load_model(f)
        print(f"Loaded efficientNet_model_b0 from {f}")

    elif 'b5' in fname:
        efficientNet_model_b5 = load_model(f)
        print(f"Loaded efficientNet_model_b5 from {f}")

    elif 'dense' in fname or 'densenet121' in fname:
        densenet121_model = load_model(f)
        print(f"Loaded densenet121_model from {f}")

    elif 'custom' in fname or 'custom_cnn' in fname:
        custom_cnn_model = load_model(f)
        print(f"Loaded custom_cnn_model from {f}")

print("\nDone loading available models.")


# -----------------------------
# 2) LOAD SAVED HISTORIES
# -----------------------------

class LoadedHistory:
    """
    This makes loaded histories behave like Keras history objects.
    Example:
        efficientnet_history.history['accuracy']
    """
    def __init__(self, history_dict):
        self.history = history_dict


history_files = []

for root, dirs, files in os.walk(base_path):
    for file in files:
        fname = file.lower()

        if 'history' in fname and (
            fname.endswith('.csv') or
            fname.endswith('.json') or
            fname.endswith('.pkl') or
            fname.endswith('.pickle')
        ):
            history_files.append(os.path.join(root, file))

print("\nFound history files:")
for f in history_files:
    print(f)


def load_history_file(path):
    fname = path.lower()

    if fname.endswith('.csv'):
        df = pd.read_csv(path)
        return LoadedHistory(df.to_dict(orient='list'))

    elif fname.endswith('.json'):
        with open(path, 'r') as f:
            history_dict = json.load(f)
        return LoadedHistory(history_dict)

    elif fname.endswith('.pkl') or fname.endswith('.pickle'):
        with open(path, 'rb') as f:
            history_dict = pickle.load(f)
        return LoadedHistory(history_dict)

    else:
        return None


for f in history_files:
    fname = os.path.basename(f).lower()
    loaded_history = load_history_file(f)

    if loaded_history is None:
        continue

    if 'b0r' in fname or 'efficientnet_b0r_history' in fname:
        efficientnet_b0R_history = loaded_history
```

```
            print(f"Loaded efficientnet_b0R_history from {f}")

        elif 'b5' in fname or 'efficientnet_history_b5' in fname:
            efficientnet_history_b5 = loaded_history
            print(f"Loaded efficientnet_history_b5 from {f}")

        elif 'dense' in fname or 'densenet_history' in fname:
            DenseNet_history = loaded_history
            print(f"Loaded DenseNet_history from {f}")

        elif 'custom' in fname or 'custom_cnn_history' in fname:
            custom_cnn_history = loaded_history
            print(f"Loaded custom_cnn_history from {f}")

        elif 'efficientnet_history' in fname or 'b0' in fname:
            efficientnet_history = loaded_history
            print(f"Loaded efficientnet_history from {f}")

    print("\nDone loading available histories.")
```

Mostra output nascosti

5. Model Comparison and Best Model Selection

At this stage, the notebook compares the trained models using the same evaluation pipeline.

What this section is trying to decide

The goal is not only to find the model with the highest score, but also to identify the most reasonable candidate for the next step of the project. A good selection should consider:

- validation behavior,
- test accuracy,
- training stability,
- architectural complexity.

This makes the model-selection decision more defensible than choosing a model by a single metric alone.

Interpretation note

The comparison tables and plots should be read as part of a decision process:

1. identify the strongest baseline,
2. check whether the accuracy gain is meaningful,
3. decide whether the extra complexity is justified,
4. choose one model for tuning.

This makes the workflow resemble a small experimental study rather than a sequence of isolated training runs.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import f1_score

def get_best_history_metrics(history):
    best_val_acc = max(history.history['val_accuracy'])
    best_val_loss = min(history.history['val_loss'])
    final_train_acc = history.history['accuracy'][-1]
    final_train_loss = history.history['loss'][-1]
    best_epoch = np.argmax(history.history['val_accuracy']) + 1
    return best_val_acc, best_val_loss, final_train_acc, final_train_loss, best_epoch

# Evaluate all models on the test set
results = []

models_to_compare = [
    ("EfficientNetB0_Original", efficientNet_model_b0, efficientnet_history),
    ("EfficientNetB0_Simplified", efficientNet_model_b0R, efficientnet_b0R_history),
    ("EfficientNetB5", efficientNet_model_b5, efficientnet_history_b5),
    ("DenseNet121", densenet121_model, DenseNet_history),
    ("Custom_CNN", custom_cnn_model, custom_cnn_history)
]

for model_name, model_obj, history_obj in models_to_compare:
    # Evaluate on test set
    test_loss, test_acc = model_obj.evaluate(test_gen, verbose=0)
```

```
    # NEW: Compute Macro F1 on the test set
    test_gen.reset()
    y_pred_probs = model_obj.predict(test_gen, verbose=0)
    y_pred = np.argmax(y_pred_probs, axis=1)
    y_true = test_gen.classes

    macro_f1 = f1_score(y_true, y_pred, average="macro")

    # Best validation metrics from history
    best_val_acc, best_val_loss, final_train_acc, final_train_loss, best_epoch = get_best_history_metrics(history_obj)

    # Store total params
    total_params = model_obj.count_params()

    results.append({
        "Model": model_name,
        "Total_Params": total_params,
        "Final_Train_Accuracy": final_train_acc,
        "Final_Train_Loss": final_train_loss,
        "Best_Val_Accuracy": best_val_acc,
        "Best_Val_Loss": best_val_loss,
        "Best_Epoch": best_epoch,
        "Test_Accuracy": test_acc,
        "Test_Loss": test_loss,
        "Macro_F1": macro_f1
    })

comparison_df = pd.DataFrame(results)

# Sort by test accuracy first, then validation accuracy
comparison_df = comparison_df.sort_values(
    by=["Test_Accuracy", "Best_Val_Accuracy"],
    ascending=False
).reset_index(drop=True)

print("Model comparison table:")
display(comparison_df)

best_model_name = comparison_df.loc[0, "Model"]
print(f"\nBest model based on test accuracy: {best_model_name}")
```

```
WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data
WARNING:tensorflow:5 out of the last 13 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data
Model comparison table:
```

	Model	Total_Params	Final_Train_Accuracy	Final_Train_Loss	Best_Val_Accuracy	Best_Val_Loss	Best_Epoch
0	EfficientNetB5	29041926	0.978612	2.560768	0.919926	2.726045	10
1	DenseNet121	7303759	0.985814	1.468065	0.944134	1.638412	20
2	EfficientNetB0_Original	4381362	0.943256	2.396245	0.929236	2.392464	15
3	EfficientNetB0_Simplified	4215474	0.818202	0.580544	0.824953	0.614474	26
4	Custom_CNN	4854127	0.849880	0.512712	0.565298	1.285370	20

```
Best model based on test accuracy: EfficientNetB5
```

```
import matplotlib.pyplot as plt

# Plot Test Accuracy comparison in decreasing order
plot_df = comparison_df.sort_values(by="Test_Accuracy", ascending=False)

plt.figure(figsize=(10, 6))
plt.bar(plot_df["Model"], plot_df["Test_Accuracy"])
plt.title("Test Accuracy Comparison of All Models")
plt.ylabel("Test Accuracy")
plt.xticks(rotation=20)
plt.ylim(0, 1)

for i, v in enumerate(plot_df["Test_Accuracy"]):
    plt.text(i, v + 0.01, f"{v:.4f}", ha='center')

plt.tight_layout()
plt.show()


# Plot Best Validation Accuracy comparison in decreasing order
plot_df = comparison_df.sort_values(by="Best_Val_Accuracy", ascending=False)

plt.figure(figsize=(10, 6))
```

```
plt.bar(plot_df["Model"], plot_df["Best_Val_Accuracy"])
plt.title("Best Validation Accuracy Comparison of All Models")
plt.ylabel("Best Validation Accuracy")
plt.xticks(rotation=20)
plt.ylim(0, 1)

for i, v in enumerate(plot_df["Best_Val_Accuracy"]):
    plt.text(i, v + 0.01, f"{v:.4f}", ha='center')

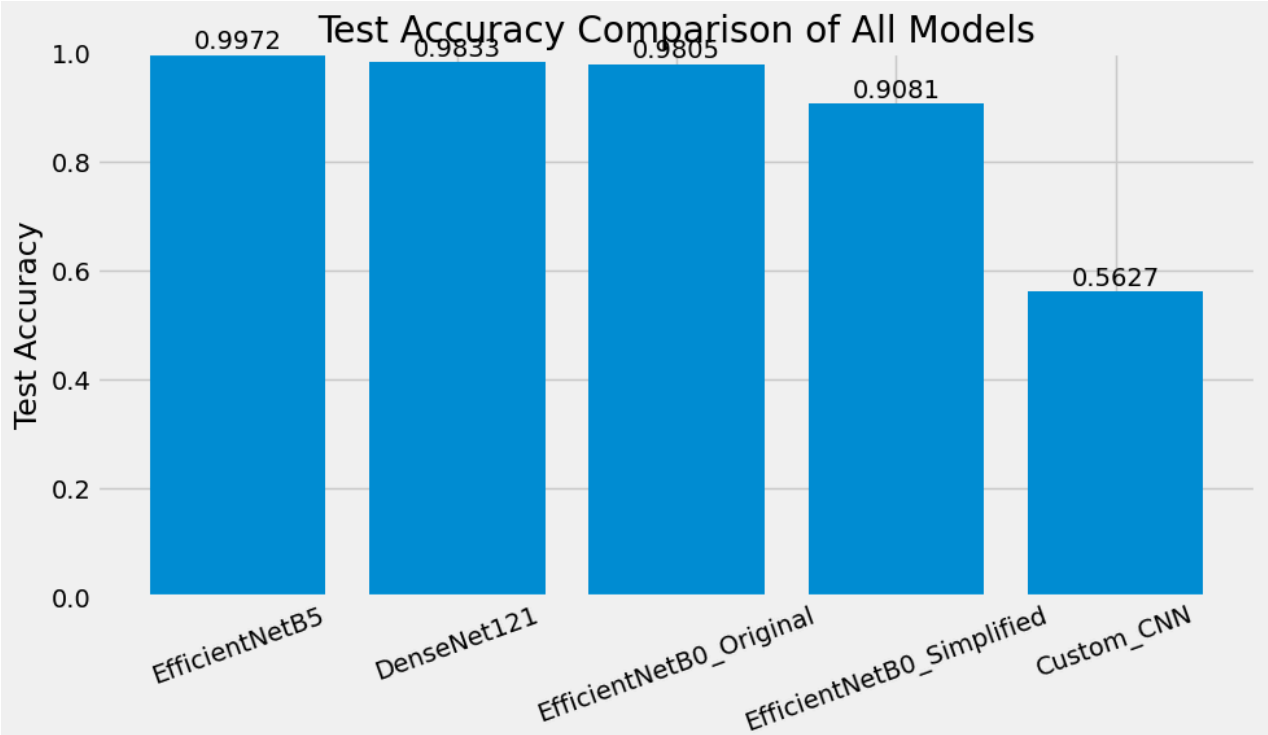
plt.tight_layout()
plt.show()

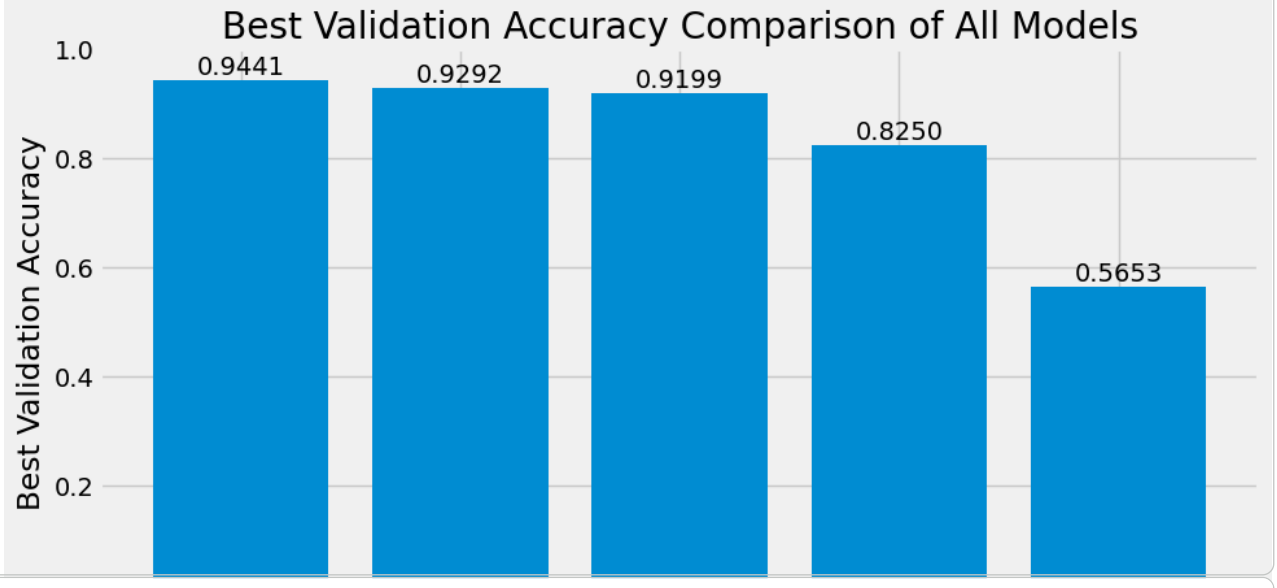

# Plot Macro F1 comparison in decreasing order
plot_df = comparison_df.sort_values(by="Macro_F1", ascending=False)

plt.figure(figsize=(10, 6))
plt.bar(plot_df["Model"], plot_df["Macro_F1"])
plt.title("Macro F1 Score Comparison of All Models")
plt.ylabel("Macro F1 Score")
plt.xticks(rotation=20)
plt.ylim(0, 1)

for i, v in enumerate(plot_df["Macro_F1"]):
    plt.text(i, v + 0.01, f"{v:.4f}", ha='center')

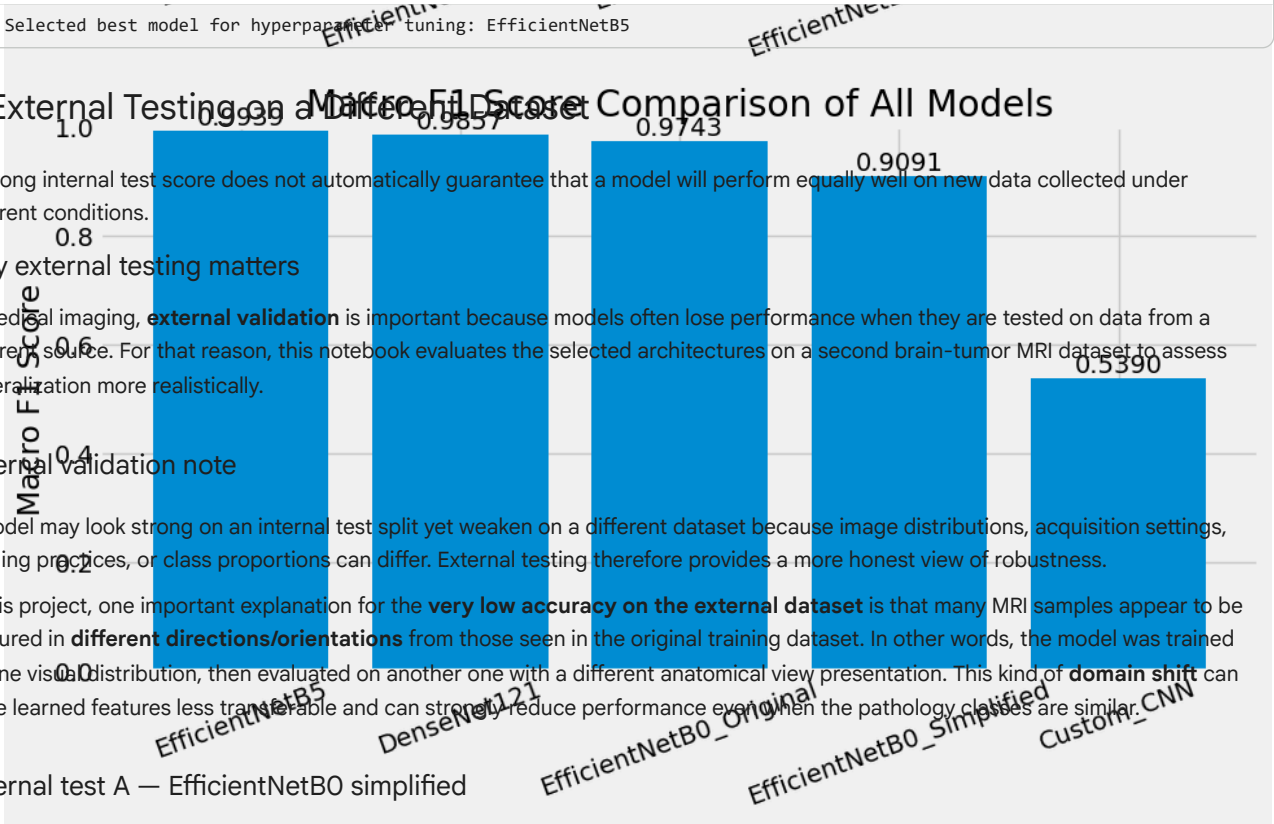
plt.tight_layout()
plt.show()
```





```
best_model_name = comparison_df.loc[0, "Model"]
print("Selected best model for hyperparameter tuning:", best_model_name)
```

```
Selected best model for hyperparameter tuning: EfficientNetB5
```



7. External Testing on a Different Dataset

A strong internal test score does not automatically guarantee that a model will perform equally well on new data collected under different conditions.

Why external testing matters

In medical imaging, **external validation** is important because models often lose performance when they are tested on data from a different source. For that reason, this notebook evaluates the selected architectures on a second brain-tumor MRI dataset to assess generalization more realistically.

External validation note

A model may look strong on an internal test split yet weaken on a different dataset because image distributions, acquisition settings, labeling practices, or class proportions can differ. External testing therefore provides a more honest view of robustness.

In this project, one important explanation for the **very low accuracy on the external dataset** is that many MRI samples appear to be captured in **different directions/orientations** from those seen in the original training dataset. In other words, the model was trained on one visual distribution, then evaluated on another one with a different anatomical view presentation. This kind of **domain shift** can make learned features less transferable and can strongly reduce performance even when the pathology classes are similar.

External test A — EfficientNetB0 simplified

The following block evaluates the simplified EfficientNetB0 model on a separate external dataset. This helps determine whether the lighter task-specific head remains robust outside the original data split.

```
import os
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from tensorflow.keras.preprocessing.image import ImageDataGenerator



model_to_test = efficientNet_model_b0R
model_name = "EfficientNetB0_Simplified"


path = kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-dataset")
print("Dataset downloaded to:", path)

possible_folders = [
    os.path.join(path, "Testing"),
    os.path.join(path, "Training"),
    path
]

external_dataset_root = None
for folder in possible_folders:
    if os.path.exists(folder):
        subdirs = [d for d in os.listdir(folder) if os.path.isdir(os.path.join(folder, d))]
        if len(subdirs) > 0:
            external_dataset_root = folder
            break

if external_dataset_root is None:
    raise FileNotFoundError("Could not find a valid external dataset folder.")

print("Using external dataset folder:", external_dataset_root)


original_class_names = list(train_gen.class_indices.keys())
print("\nOriginal training classes:")
```

```
print(train_gen.class_indices)


new_to_original_label_map = {
    "glioma": "Glioblastoma",
    "meningioma": "Meningioma",
    "notumor": "_NORMAL"
}

print("\nRequested label mapping:")
print(new_to_original_label_map)

filtered_label_map = {}
for new_label, old_label in new_to_original_label_map.items():
    if old_label in original_class_names:
        filtered_label_map[new_label] = old_label
    else:
        print(f"Skipping mapping '{new_label} -> {old_label}' because '{old_label}' is not in the original training classes

if len(filtered_label_map) == 0:
    raise ValueError("No valid label mappings found between the external dataset and the original training classes.")

print("\nValid label mapping used:")
print(filtered_label_map)


filepaths_external = []
labels_external = []

for folder_name in os.listdir(external_dataset_root):
    folder_path = os.path.join(external_dataset_root, folder_name)

    if not os.path.isdir(folder_path):
        continue

    if folder_name not in filtered_label_map:
        print(f"Skipping folder '{folder_name}' because it is not mapped.")
        continue

    mapped_label = filtered_label_map[folder_name]

    for fname in os.listdir(folder_path):
        fpath = os.path.join(folder_path, fname)
        if os.path.isfile(fpath) and fname.lower().endswith((".jpg", ".jpeg", ".png", ".bmp", ".webp")):
            filepaths_external.append(fpath)
            labels_external.append(mapped_label)

external_df = pd.DataFrame({
    "filepaths": filepaths_external,
    "labels": labels_external
})

if len(external_df) == 0:
    raise ValueError("No images found for the mapped classes in the external dataset.")

print("\nExternal dataset size after mapping:", len(external_df))
print("\nExternal label distribution:")
print(external_df["labels"].value_counts())

display(external_df.head())


external_datagen = ImageDataGenerator()

external_test_gen = external_datagen.flow_from_dataframe(
    dataframe=external_df,
    x_col="filepaths",
    y_col="labels",
    target_size=img_size,
    color_mode="rgb",
    class_mode="categorical",
    batch_size=batch_size,
    shuffle=False,
    classes=original_class_names
)

print("\nExternal generator class indices:")
print(external_test_gen.class_indices)


def evaluate_model_on_external_dataset_3class(model, model_name, generator):
    print("\n" + "="*70)
```

```
    print(f"Evaluating {model_name} on external dataset (3-class view)")
    print("="*70)

    steps = math.ceil(generator.samples / generator.batch_size)

    target_classes = ["Glioblastoma", "Meningioma", "_NORMAL"]

    for c in target_classes:
        if c not in generator.class_indices:
            raise ValueError(f"Class '{c}' is not present in generator.class_indices. Check your mapping.")

    target_ids = [generator.class_indices[c] for c in target_classes]

    print("\nUsing only these output classes:")
    for c, i in zip(target_classes, target_ids):
        print(f"{c}: class id {i}")

    generator.reset()
    y_pred_probs = model.predict(generator, steps=steps, verbose=1)

    y_true_full = np.array(generator.classes)

    keep_mask = np.isin(y_true_full, target_ids)
    y_true_full = y_true_full[keep_mask]
    y_pred_probs = y_pred_probs[keep_mask]

    y_pred_probs_3 = y_pred_probs[:, target_ids]
    y_pred_3 = np.argmax(y_pred_probs_3, axis=1)

    id_to_3class_index = {orig_id: new_idx for new_idx, orig_id in enumerate(target_ids)}
    y_true_3 = np.array([id_to_3class_index[y] for y in y_true_full])

    acc = accuracy_score(y_true_3, y_pred_3)
    print(f"\n{model_name} - 3-Class External Accuracy: {acc:.4f}")

    print(f"\nClassification Report - {model_name}:\n")
    print(
        classification_report(
            y_true_3,
            y_pred_3,
            labels=[0, 1, 2],
            target_names=target_classes,
            digits=4,
            zero_division=0
        )
    )

    cm = confusion_matrix(y_true_3, y_pred_3, labels=[0, 1, 2])

    plt.figure(figsize=(6, 5))
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=target_classes,
        yticklabels=target_classes
    )
    plt.title(f"Confusion Matrix - {model_name} (3-Class External)")
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.tight_layout()
    plt.show()

    pred_counts = pd.Series(y_pred_3).value_counts().sort_index()
    print("\nPredicted class distribution (3-class view):")
    for idx, count in pred_counts.items():
        print(f"{target_classes[idx]}: {count}")

    return {
        "Model": model_name,
        "3-Class External Accuracy": acc
    }


external_result = evaluate_model_on_external_dataset_3class(
    model=model_to_test,
    model_name=model_name,
    generator=external_test_gen
)
```

```
external_result_df = pd.DataFrame([external_result])

print("\nExternal testing result:")
display(external_result_df)

plt.figure(figsize=(7, 5))
sns.barplot(data=external_result_df, x="Model", y="3-Class External Accuracy")
plt.title("External Dataset Accuracy - Simplified EfficientNetB0")
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```

```
Using Colab cache for faster access to the 'brain-tumor-mri-dataset' dataset.
Dataset downloaded to: /kaggle/input/brain-tumor-mri-dataset
Using external dataset folder: /kaggle/input/brain-tumor-mri-dataset/Testing

Original training classes:
{'Astrocitoma': 0, 'Carcinoma': 1, 'Ependimoma': 2, 'Ganglioglioma': 3, 'Germinoma': 4, 'Glioblastoma': 5, 'Granuloma': 6,

Requested label mapping:
{'glioma': 'Glioblastoma', 'meningioma': 'Meningioma', 'notumor': '_NORMAL'}

Valid label mapping used:
{'glioma': 'Glioblastoma', 'meningioma': 'Meningioma', 'notumor': '_NORMAL'}
Skipping folder 'pituitary' because it is not mapped.

External dataset size after mapping: 1200

External label distribution:
labels
_NORMAL         400
Meningioma      400
Glioblastoma    400
Name: count, dtype: int64
```

	filepaths	labels
0	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
1	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
2	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
3	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
4	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL

```
Found 1200 validated image filenames belonging to 15 classes.

External generator class indices:
{'Astrocitoma': 0, 'Carcinoma': 1, 'Ependimoma': 2, 'Ganglioglioma': 3, 'Germinoma': 4, 'Glioblastoma': 5, 'Granuloma': 6,

======================================================================
Evaluating EfficientNetB0 simplified on external dataset (3-class view)
======================================================================
Using only these output classes:
Glioblastoma: class id 5
Meningioma: class id 8
_NORMAL: class id 11
```

External test B — EfficientNetB0 baseline

This block evaluates the baseline EfficientNetB0 model on the same external dataset so that its generalization can be compared fairly against the simplified variant and the larger backbones.

```
import os
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from tensorflow.keras.preprocessing.image import ImageDataGenerator


model_to_test = efficientNet_model_b0
model_name = "EfficientNetB0"


path = kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-dataset")
print("Dataset downloaded to:", path)

possible_folders = [
    os.path.join(path, "Testing"),
    os.path.join(path, "Training"),
    path
]

external_dataset_root = None
for folder in possible_folders:
    if os.path.exists(folder):
        subdirs = [d for d in os.listdir(folder) if os.path.isdir(os.path.join(folder, d))]
        if len(subdirs) > 0:
            external_dataset_root = folder
            break

if external_dataset_root is None:
    raise FileNotFoundError("Could not find a valid external dataset folder.")

print("Using external dataset folder:", external_dataset_root)

original_class_names = list(train_gen.class_indices.keys())
```

```
print("\nOriginal training classes:")
print(train_gen.class_indices)


new_to_original_label_map = {
    "glioma": "Glioblastoma",
    "meningioma": "Meningioma",
    "notumor": "_NORMAL"
}

print("\nRequested label mapping:")
print(new_to_original_label_map)

filtered_label_map = {}
for new_label, old_label in new_to_original_label_map.items():
    if old_label in original_class_names:
        filtered_label_map[new_label] = old_label
    else:
        print(f"Skipping mapping '{new_label} -> {old_label}' because '{old_label}' is not in the original training classes

if len(filtered_label_map) == 0:
    raise ValueError("No valid label mappings found between the external dataset and the original training classes.")

print("\nValid label mapping used:")
print(filtered_label_map)


filepaths_external = []
labels_external = []

for folder_name in os.listdir(external_dataset_root):
    folder_path = os.path.join(external_dataset_root, folder_name)

    if not os.path.isdir(folder_path):
        continue

    if folder_name not in filtered_label_map:
        print(f"Skipping folder '{folder_name}' because it is not mapped.")
        continue

    mapped_label = filtered_label_map[folder_name]

    for fname in os.listdir(folder_path):
        fpath = os.path.join(folder_path, fname)
        if os.path.isfile(fpath) and fname.lower().endswith((".jpg", ".jpeg", ".png", ".bmp", ".webp")):
            filepaths_external.append(fpath)
            labels_external.append(mapped_label)

external_df = pd.DataFrame({
    "filepaths": filepaths_external,
    "labels": labels_external
})

if len(external_df) == 0:
    raise ValueError("No images found for the mapped classes in the external dataset.")

print("\nExternal dataset size after mapping:", len(external_df))
print("\nExternal label distribution:")
print(external_df["labels"].value_counts())

display(external_df.head())

external_datagen = ImageDataGenerator()

external_test_gen = external_datagen.flow_from_dataframe(
    dataframe=external_df,
    x_col="filepaths",
    y_col="labels",
    target_size=img_size,
    color_mode="rgb",
    class_mode="categorical",
    batch_size=batch_size,
    shuffle=False,
    classes=original_class_names
)

print("\nExternal generator class indices:")
print(external_test_gen.class_indices)


def evaluate_model_on_external_dataset_3class(model, model_name, generator):
    print("\n" + "="*70)
```

```
    print(f"Evaluating {model_name} on external dataset (3-class view)")
    print("="*70)

    steps = math.ceil(generator.samples / generator.batch_size)

    target_classes = ["Glioblastoma", "Meningioma", "_NORMAL"]

    for c in target_classes:
        if c not in generator.class_indices:
            raise ValueError(f"Class '{c}' is not present in generator.class_indices. Check your mapping.")

    target_ids = [generator.class_indices[c] for c in target_classes]

    print("\nUsing only these output classes:")
    for c, i in zip(target_classes, target_ids):
        print(f"{c}: class id {i}")

    generator.reset()
    y_pred_probs = model.predict(generator, steps=steps, verbose=1)

    y_true_full = np.array(generator.classes)

    keep_mask = np.isin(y_true_full, target_ids)
    y_true_full = y_true_full[keep_mask]
    y_pred_probs = y_pred_probs[keep_mask]

    y_pred_probs_3 = y_pred_probs[:, target_ids]
    y_pred_3 = np.argmax(y_pred_probs_3, axis=1)

    id_to_3class_index = {orig_id: new_idx for new_idx, orig_id in enumerate(target_ids)}
    y_true_3 = np.array([id_to_3class_index[y] for y in y_true_full])

    acc = accuracy_score(y_true_3, y_pred_3)
    print(f"\n{model_name} - 3-Class External Accuracy: {acc:.4f}")

    print(f"\nClassification Report - {model_name}:\n")
    print(
        classification_report(
            y_true_3,
            y_pred_3,
            labels=[0, 1, 2],
            target_names=target_classes,
            digits=4,
            zero_division=0
        )
    )

    cm = confusion_matrix(y_true_3, y_pred_3, labels=[0, 1, 2])

    plt.figure(figsize=(6, 5))
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=target_classes,
        yticklabels=target_classes
    )
    plt.title(f"Confusion Matrix - {model_name} (3-Class External)")
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.tight_layout()
    plt.show()

    pred_counts = pd.Series(y_pred_3).value_counts().sort_index()
    print("\nPredicted class distribution (3-class view):")
    for idx, count in pred_counts.items():
        print(f"{target_classes[idx]}: {count}")

    return {
        "Model": model_name,
        "3-Class External Accuracy": acc
    }


external_result = evaluate_model_on_external_dataset_3class(
    model=model_to_test,
    model_name=model_name,
    generator=external_test_gen
)
```

```
external_result_df = pd.DataFrame([external_result])

print("\nExternal testing result:")
display(external_result_df)

plt.figure(figsize=(7, 5))
sns.barplot(data=external_result_df, x="Model", y="3-Class External Accuracy")
plt.title("External Dataset Accuracy -  EfficientNetB0")
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```

```
Using Colab cache for faster access to the 'brain-tumor-mri-dataset' dataset.
Dataset downloaded to: /kaggle/input/brain-tumor-mri-dataset
Using external dataset folder: /kaggle/input/brain-tumor-mri-dataset/Testing

Original training classes:
{'Astrocitoma': 0, 'Carcinoma': 1, 'Ependimoma': 2, 'Ganglioglioma': 3, 'Germinoma': 4, 'Glioblastoma': 5, 'Granuloma': 6,

Requested label mapping:
{'glioma': 'Glioblastoma', 'meningioma': 'Meningioma', 'notumor': '_NORMAL'}

Valid label mapping used:
{'glioma': 'Glioblastoma', 'meningioma': 'Meningioma', 'notumor': '_NORMAL'}
Skipping folder 'pituitary' because it is not mapped.

External dataset size after mapping: 1200

External label distribution:
labels
_NORMAL         400
Meningioma      400
Glioblastoma    400
Name: count, dtype: int64
```

	filepaths	labels
0	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
1	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
2	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
3	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
4	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL

```
Found 1200 validated image filenames belonging to 15 classes.

External generator class indices:
{'Astrocitoma': 0, 'Carcinoma': 1, 'Ependimoma': 2, 'Ganglioglioma': 3, 'Germinoma': 4, 'Glioblastoma': 5, 'Granuloma': 6,

======================================================================
Evaluating EfficientNetB5 on external dataset (3-class view)
======================================================================
```

External test C — EfficientNetB5

```
import os
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from tensorflow.keras.preprocessing.image import ImageDataGenerator



model_to_test = efficientNet_model_b5
model_name = "EfficientNetB5"


path = kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-dataset")
print("Dataset downloaded to:", path)

possible_folders = [
    os.path.join(path, "Testing"),
    os.path.join(path, "Training"),
    path
]

external_dataset_root = None
for folder in possible_folders:
    if os.path.exists(folder):
        subdirs = [d for d in os.listdir(folder) if os.path.isdir(os.path.join(folder, d))]
        if len(subdirs) > 0:
            external_dataset_root = folder
            break

if external_dataset_root is None:
    raise FileNotFoundError("Could not find a valid external dataset folder.")

print("Using external dataset folder:", external_dataset_root)


original_class_names = list(train_gen.class_indices.keys())
print("\nOriginal training classes:")
```

```
print(train_gen.class_indices)


new_to_original_label_map = {
    "glioma": "Glioblastoma",
    "meningioma": "Meningioma",
    "notumor": "_NORMAL"
}

print("\nRequested label mapping:")
print(new_to_original_label_map)

filtered_label_map = {}
for new_label, old_label in new_to_original_label_map.items():
    if old_label in original_class_names:
        filtered_label_map[new_label] = old_label
    else:
        print(f"Skipping mapping '{new_label} -> {old_label}' because '{old_label}' is not in the original training classes

if len(filtered_label_map) == 0:
    raise ValueError("No valid label mappings found between the external dataset and the original training classes.")

print("\nValid label mapping used:")
print(filtered_label_map)


filepaths_external = []
labels_external = []

for folder_name in os.listdir(external_dataset_root):
    folder_path = os.path.join(external_dataset_root, folder_name)

    if not os.path.isdir(folder_path):
        continue

    if folder_name not in filtered_label_map:
        print(f"Skipping folder '{folder_name}' because it is not mapped.")
        continue

    mapped_label = filtered_label_map[folder_name]

    for fname in os.listdir(folder_path):
        fpath = os.path.join(folder_path, fname)
        if os.path.isfile(fpath) and fname.lower().endswith((".jpg", ".jpeg", ".png", ".bmp", ".webp")):
            filepaths_external.append(fpath)
            labels_external.append(mapped_label)

external_df = pd.DataFrame({
    "filepaths": filepaths_external,
    "labels": labels_external
})

if len(external_df) == 0:
    raise ValueError("No images found for the mapped classes in the external dataset.")

print("\nExternal dataset size after mapping:", len(external_df))
print("\nExternal label distribution:")
print(external_df["labels"].value_counts())

display(external_df.head())


external_datagen = ImageDataGenerator()

external_test_gen = external_datagen.flow_from_dataframe(
    dataframe=external_df,
    x_col="filepaths",
    y_col="labels",
    target_size=img_size,
    color_mode="rgb",
    class_mode="categorical",
    batch_size=batch_size,
    shuffle=False,
    classes=original_class_names
)

print("\nExternal generator class indices:")
print(external_test_gen.class_indices)


def evaluate_model_on_external_dataset_3class(model, model_name, generator):
    print("\n" + "="*70)
```

```
        print(f"Evaluating {model_name} on external dataset (3-class view)")
        print("="*70)

        steps = math.ceil(generator.samples / generator.batch_size)

        target_classes = ["Glioblastoma", "Meningioma", "_NORMAL"]

        for c in target_classes:
            if c not in generator.class_indices:
                raise ValueError(f"Class '{c}' is not present in generator.class_indices. Check your mapping.")

        target_ids = [generator.class_indices[c] for c in target_classes]

        print("\nUsing only these output classes:")
        for c, i in zip(target_classes, target_ids):
            print(f"{c}: class id {i}")

        generator.reset()
        y_pred_probs = model.predict(generator, steps=steps, verbose=1)

        y_true_full = np.array(generator.classes)

        keep_mask = np.isin(y_true_full, target_ids)
        y_true_full = y_true_full[keep_mask]
        y_pred_probs = y_pred_probs[keep_mask]

        y_pred_probs_3 = y_pred_probs[:, target_ids]
        y_pred_3 = np.argmax(y_pred_probs_3, axis=1)

        id_to_3class_index = {orig_id: new_idx for new_idx, orig_id in enumerate(target_ids)}
        y_true_3 = np.array([id_to_3class_index[y] for y in y_true_full])

        acc = accuracy_score(y_true_3, y_pred_3)
        print(f"\n{model_name} - 3-Class External Accuracy: {acc:.4f}")

        print(f"\nClassification Report - {model_name}:\n")
        print(
            classification_report(
                y_true_3,
                y_pred_3,
                labels=[0, 1, 2],
                target_names=target_classes,
                digits=4,
                zero_division=0
            )
        )

        cm = confusion_matrix(y_true_3, y_pred_3, labels=[0, 1, 2])

        plt.figure(figsize=(6, 5))
        sns.heatmap(
            cm,
            annot=True,
            fmt="d",
            cmap="Blues",
            xticklabels=target_classes,
            yticklabels=target_classes
        )
        plt.title(f"Confusion Matrix - {model_name} (3-Class External)")
        plt.xlabel("Predicted Label")
        plt.ylabel("True Label")
        plt.tight_layout()
        plt.show()

        pred_counts = pd.Series(y_pred_3).value_counts().sort_index()
        print("\nPredicted class distribution (3-class view):")
        for idx, count in pred_counts.items():
            print(f"{target_classes[idx]}: {count}")

        return {
            "Model": model_name,
            "3-Class External Accuracy": acc
        }

external_result = evaluate_model_on_external_dataset_3class(
    model=model_to_test,
    model_name=model_name,
    generator=external_test_gen
)


external_result_df = pd.DataFrame([external_result])
```

```
print("\nExternal testing result:")
display(external_result_df)

plt.figure(figsize=(7, 5))
sns.barplot(data=external_result_df, x="Model", y="3-Class External Accuracy")
plt.title("External Dataset Accuracy - Simplified EfficientNetB0")
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```

```
Using Colab cache for faster access to the 'brain-tumor-mri-dataset' dataset.
Dataset downloaded to: /kaggle/input/brain-tumor-mri-dataset
Using external dataset folder: /kaggle/input/brain-tumor-mri-dataset/Testing

Original training classes:
{'Astrocitoma': 0, 'Carcinoma': 1, 'Ependimoma': 2, 'Ganglioglioma': 3, 'Germinoma': 4, 'Glioblastoma': 5, 'Granuloma': 6,

Requested label mapping:
{'glioma': 'Glioblastoma', 'meningioma': 'Meningioma', 'notumor': '_NORMAL'}

Valid label mapping used:
{'glioma': 'Glioblastoma', 'meningioma': 'Meningioma', 'notumor': '_NORMAL'}
Skipping folder 'pituitary' because it is not mapped.

External dataset size after mapping: 1200

External label distribution:
labels
_NORMAL         400
Meningioma      400
Glioblastoma    400
Name: count, dtype: int64
```

	filepaths	labels
0	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
1	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
2	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
3	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL
4	/kaggle/input/brain-tumor-mri-dataset/Testing/...	_NORMAL

```
Found 1200 validated image filenames belonging to 15 classes.

External generator class indices:
{'Astrocitoma': 0, 'Carcinoma': 1, 'Ependimoma': 2, 'Ganglioglioma': 3, 'Germinoma': 4, 'Glioblastoma': 5, 'Granuloma': 6,

======================================================================
Evaluating EfficientNetB3 on external dataset (3-class view)
======================================================================

Using only these output classes:
Glioblastoma: class id 5
Meningioma: class id 8
_NORMAL: class id 14
19/19 ━━━━━━━━━━━━━━━━━━━━ 27s 1s/step
```

External test D — DenseNet121

↳ 1 cella nascosta

custom

```
import os
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from tensorflow.keras.preprocessing.image import ImageDataGenerator


model_to_test = custom_cnn_model
model_name = "custom_cnn"


path = kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-dataset")
print("Dataset downloaded to:", path)

possible_folders = [
    os.path.join(path, "Testing"),
    os.path.join(path, "Training"),
    path
]

external_dataset_root = None
for folder in possible_folders:
    if os.path.exists(folder):
        subdirs = [d for d in os.listdir(folder) if os.path.isdir(os.path.join(folder, d))]
        if len(subdirs) > 0:
            external_dataset_root = folder
            break

if external_dataset_root is None:
    raise FileNotFoundError("Could not find a valid external dataset folder.")

print("Using external dataset folder:", external_dataset_root)
```

```
original_class_names = list(train_gen.class_indices.keys())
print("\nOriginal training classes:")
print(train_gen.class_indices)


new_to_original_label_map = {
    "glioma": "Glioblastoma",
    "meningioma": "Meningioma",
    "notumor": "_NORMAL"
}

print("\nRequested label mapping:")
print(new_to_original_label_map)

filtered_label_map = {}
for new_label, old_label in new_to_original_label_map.items():
    if old_label in original_class_names:
        filtered_label_map[new_label] = old_label
    else:
        print(f"Skipping mapping '{new_label} -> {old_label}' because '{old_label}' is not in the original training classes

if len(filtered_label_map) == 0:
    raise ValueError("No valid label mappings found between the external dataset and the original training classes.")

print("\nValid label mapping used:")
print(filtered_label_map)


filepaths_external = []
labels_external = []

for folder_name in os.listdir(external_dataset_root):
    folder_path = os.path.join(external_dataset_root, folder_name)

    if not os.path.isdir(folder_path):
        continue

    if folder_name not in filtered_label_map:
        print(f"Skipping folder '{folder_name}' because it is not mapped.")
        continue

    mapped_label = filtered_label_map[folder_name]

    for fname in os.listdir(folder_path):
        fpath = os.path.join(folder_path, fname)
        if os.path.isfile(fpath) and fname.lower().endswith((".jpg", ".jpeg", ".png", ".bmp", ".webp")):
            filepaths_external.append(fpath)
            labels_external.append(mapped_label)

external_df = pd.DataFrame({
    "filepaths": filepaths_external,
    "labels": labels_external
})

if len(external_df) == 0:
    raise ValueError("No images found for the mapped classes in the external dataset.")

print("\nExternal dataset size after mapping:", len(external_df))
print("\nExternal label distribution:")
print(external_df["labels"].value_counts())

display(external_df.head())


external_datagen = ImageDataGenerator()

external_test_gen = external_datagen.flow_from_dataframe(
    dataframe=external_df,
    x_col="filepaths",
    y_col="labels",
    target_size=img_size,
    color_mode="rgb",
    class_mode="categorical",
    batch_size=batch_size,
    shuffle=False,
    classes=original_class_names
)

print("\nExternal generator class indices:")
print(external_test_gen.class_indices)
```

```
def evaluate_model_on_external_dataset_3class(model, model_name, generator):
    print("\n" + "="*70)
    print(f"Evaluating {model_name} on external dataset (3-class view)")
    print("="*70)

    steps = math.ceil(generator.samples / generator.batch_size)

    target_classes = ["Glioblastoma", "Meningioma", "_NORMAL"]

    for c in target_classes:
        if c not in generator.class_indices:
            raise ValueError(f"Class '{c}' is not present in generator.class_indices. Check your mapping.")

    target_ids = [generator.class_indices[c] for c in target_classes]

    print("\nUsing only these output classes:")
    for c, i in zip(target_classes, target_ids):
        print(f"{c}: class id {i}")

    generator.reset()
    y_pred_probs = model.predict(generator, steps=steps, verbose=1)

    y_true_full = np.array(generator.classes)

    keep_mask = np.isin(y_true_full, target_ids)
    y_true_full = y_true_full[keep_mask]
    y_pred_probs = y_pred_probs[keep_mask]

    y_pred_probs_3 = y_pred_probs[:, target_ids]
    y_pred_3 = np.argmax(y_pred_probs_3, axis=1)

    id_to_3class_index = {orig_id: new_idx for new_idx, orig_id in enumerate(target_ids)}
    y_true_3 = np.array([id_to_3class_index[y] for y in y_true_full])

    acc = accuracy_score(y_true_3, y_pred_3)
    print(f"\n{model_name} - 3-Class External Accuracy: {acc:.4f}")

    print(f"\nClassification Report - {model_name}:\n")
    print(
        classification_report(
            y_true_3,
            y_pred_3,
            labels=[0, 1, 2],
            target_names=target_classes,
            digits=4,
            zero_division=0
        )
    )

    cm = confusion_matrix(y_true_3, y_pred_3, labels=[0, 1, 2])

    plt.figure(figsize=(6, 5))
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=target_classes,
        yticklabels=target_classes
    )
    plt.title(f"Confusion Matrix - {model_name} (3-Class External)")
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.tight_layout()
    plt.show()

    pred_counts = pd.Series(y_pred_3).value_counts().sort_index()
    print("\nPredicted class distribution (3-class view):")
    for idx, count in pred_counts.items():
        print(f"{target_classes[idx]}: {count}")

    return {
        "Model": model_name,
        "3-Class External Accuracy": acc
    }

external_result = evaluate_model_on_external_dataset_3class(
    model=model_to_test,
    model_name=model_name,
    generator=external_test_gen
```

```
)

external_result_df = pd.DataFrame([external_result])

print("\nExternal testing result:")
display(external_result_df)

plt.figure(figsize=(7, 5))
sns.barplot(data=external_result_df, x="Model", y="3-Class External Accuracy")
plt.title("External Dataset Accuracy - Simplified EfficientNetB0")
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```